

Practical File

Cloud Computing (23CS009)

BACHELOR OF ENGINEERING

in

Computer Science and Engineering

Submitted by:

Harshita
2310990601
G-24
5th Sem
3rd Year

Submitted to:

Dr. Prabhnoor Bachhal
Assistant Professor



CHITKARA UNIVERSITY, PUNJAB
CHANDIGARH-PATIALA NATIONAL HIGHWAY
RAJPURA (PATIALA) PUNJAB-140401 (INDIA)

INDEX

Practical No.	Practical Name	Page No.
1.	Sign up for AWS, explore free-tier services, and launch EC2 instances using Ubuntu/Linux and Windows operating systems.	1 – 9
2.	Connect to an EC2 instance via EC2 Connect, SSH Client, AWS CLI, and PuTTY SSH Client using a key pair file. Access and modify Security Group inbound and outbound rules to control traffic for EC2 instances.	10 - 16
3.	Set up and configure web servers on EC2 instances, including Apache and Nginxserver. Configure auto-scaling for EC2 instances to handle variable traffic.	17 – 33
4.	Create an Amazon S3 bucket, upload objects, set up access controls with bucket policies, and use it for hosting static content.	34 - 44
5.	Set up an AWS Elastic Load Balancer (ELB) to distribute traffic among EC2 instances.	
6.	Set up Amazon VPC networking, including subnets, route tables, security groups. Scenario 1: VPC With a Public Subnet Only (Standalone Web) Scenario 2: VPC with Public and Private Subnets (3 Tier App)	
7.	Configure Amazon CloudWatch to monitor EC2 instance metrics and set up alarms.	
8.	Create an SNS, subscribe and publish a message, set up an SQS queue, integrate it with an EC2 instance, and use SES to send event notifications via email.	

Practical No. 1

Practical Title: Sign up for AWS, Explore Free-Tier Services, and Launch EC2 Instances using Ubuntu and Windows.

Objective:

1. To create an AWS account and explore free-tier services.
2. To launch an EC2 instance with Ubuntu and Windows OS.

Step 1: AWS Account Signup

1. Visit AWS Portal: Open <https://aws.amazon.com/> and click on "Create an AWS Account".
2. Enter account details, set account name, click Continue.
3. Enter billing details (AWS Free Tier requires card verification).
4. Verify via OTP sent to your mobile.
5. Select Free Tier (Basic Support) plan.
6. Sign in to AWS Console after account activation.

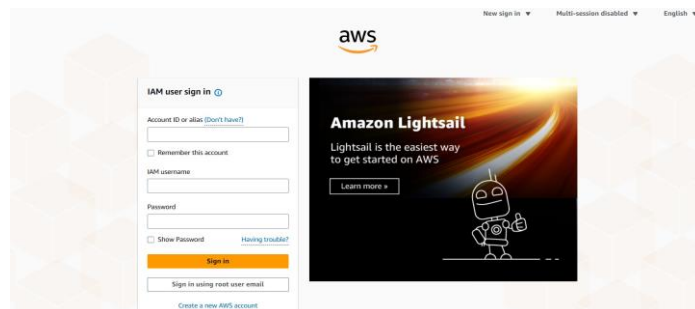


Figure 1.1

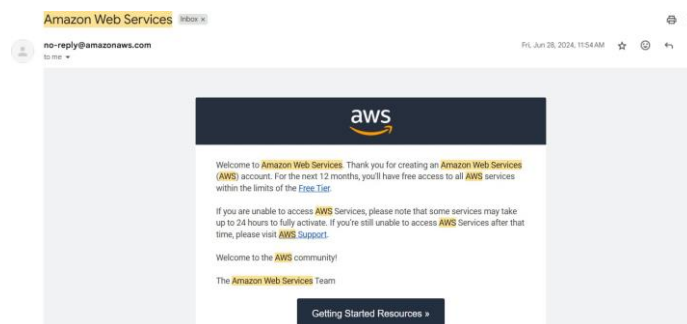


Figure 1.2

Step 2: Explore AWS Free-Tier Services

1. Navigate to AWS Console → Explore EC2, S3, Lambda, RDS, etc.
2. Check Free Tier Dashboard from Billing → Free Tier.

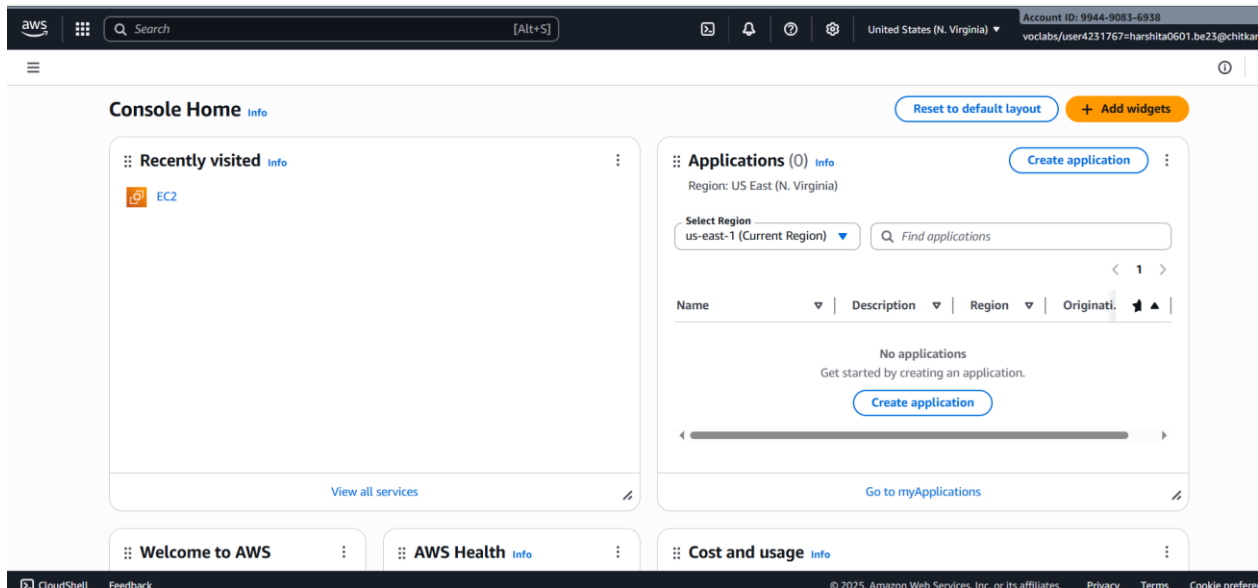


Figure 1.3

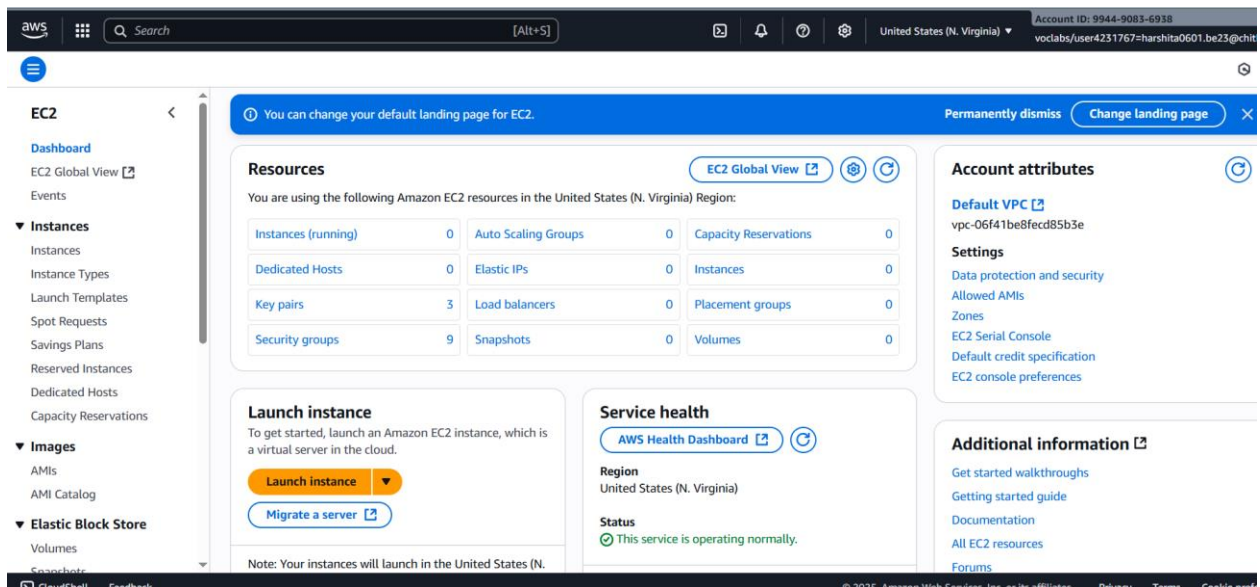


Figure 1.4

Step 3: Launch EC2 Instances

(A) Ubuntu EC2 Instance

1. Go to EC2 → Launch Instance.
2. Select AMI: Ubuntu 22.04 LTS (Free Tier).
3. Instance type: t2.micro.
4. Keep default settings, storage: 8 GB.
5. Add Name tag: Ubuntu_Instance.
6. Configure Security Group: Allow SSH (Port 22).
7. Create & download key pair (.pem).
8. Launch instance and connect via SSH:- `ssh -i my-key.pem ubuntu@ec2<your-public-ip>.compute-1.amazonaws.com`

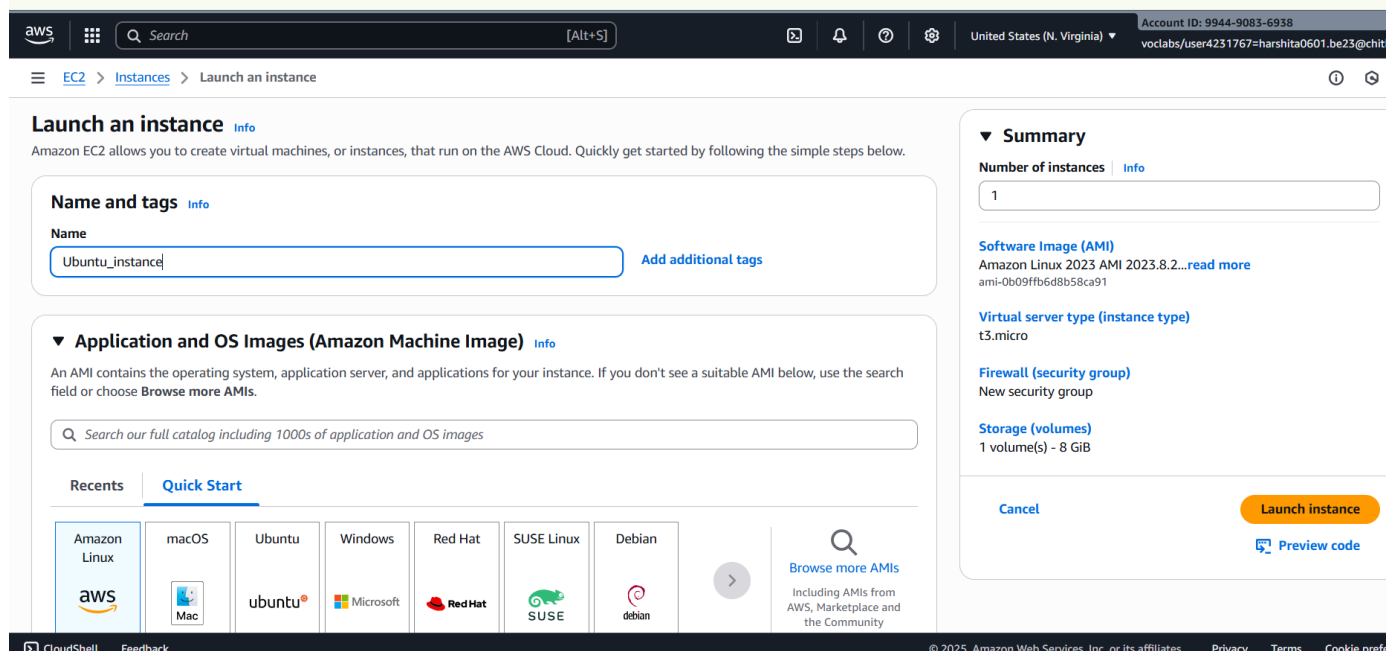


Figure 1.5

Create key pair

Key pair name
Key pairs allow you to connect to your instance securely.
ubuntu_instance
The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ RSA
RSA encrypted private and public key pair

☐ ED25519
ED25519 encrypted private and public key pair

Private key file format

☒ .pem
For use with OpenSSH

☐ .ppk
For use with PuTTY

⚠ When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. [Learn more](#)

Cancel Create key pair

Figure 1.6

Success
Successfully initiated launch of instance (i-007db956f74ff2519)

Launch log

Next Steps

What would you like to do next with this instance, for example "create alarm" or "create backup"

Create billing usage alerts

To manage costs and avoid surprise bills, set up email notifications for billing usage thresholds.

Create billing alerts

Connect to your instance

Once your instance is running, log into it from your local computer.

Connect to instance

[Learn more](#)

Connect an RDS database

Configure the connection between an EC2 instance and a database to allow traffic flow between them.

Connect an RDS database

Create a new RDS database

[Learn more](#)

Create EBS snapshot policy

Create a policy that automates the creation, retention, and deletion of EBS snapshots

Create EBS snapshot policy

Figure 1.7

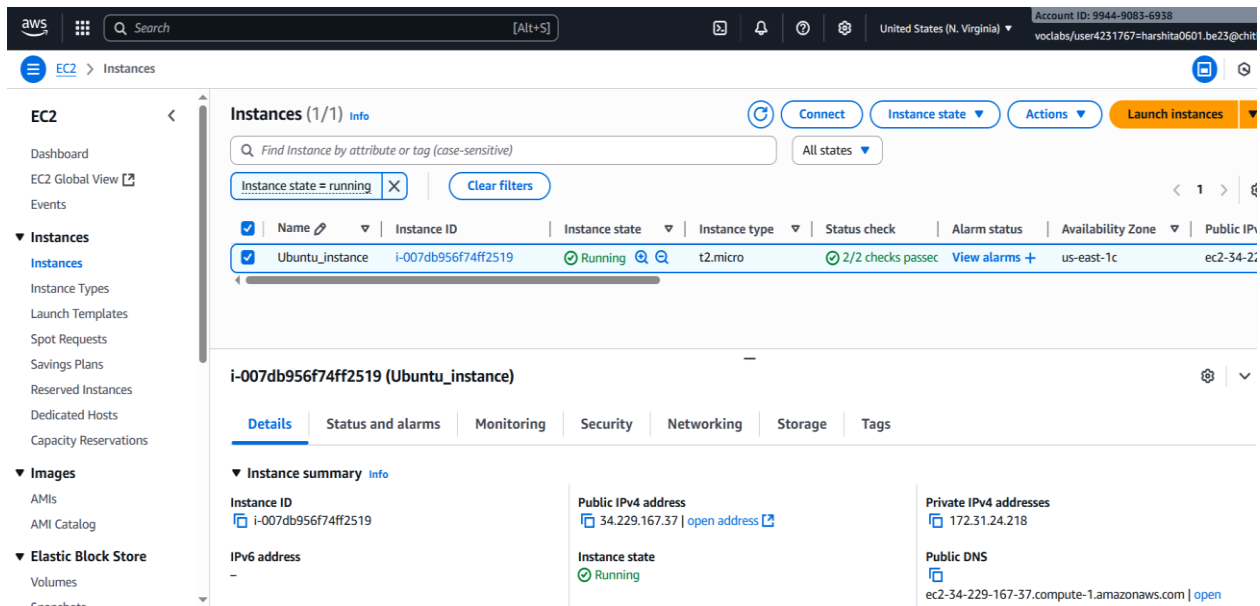


Figure 1.8

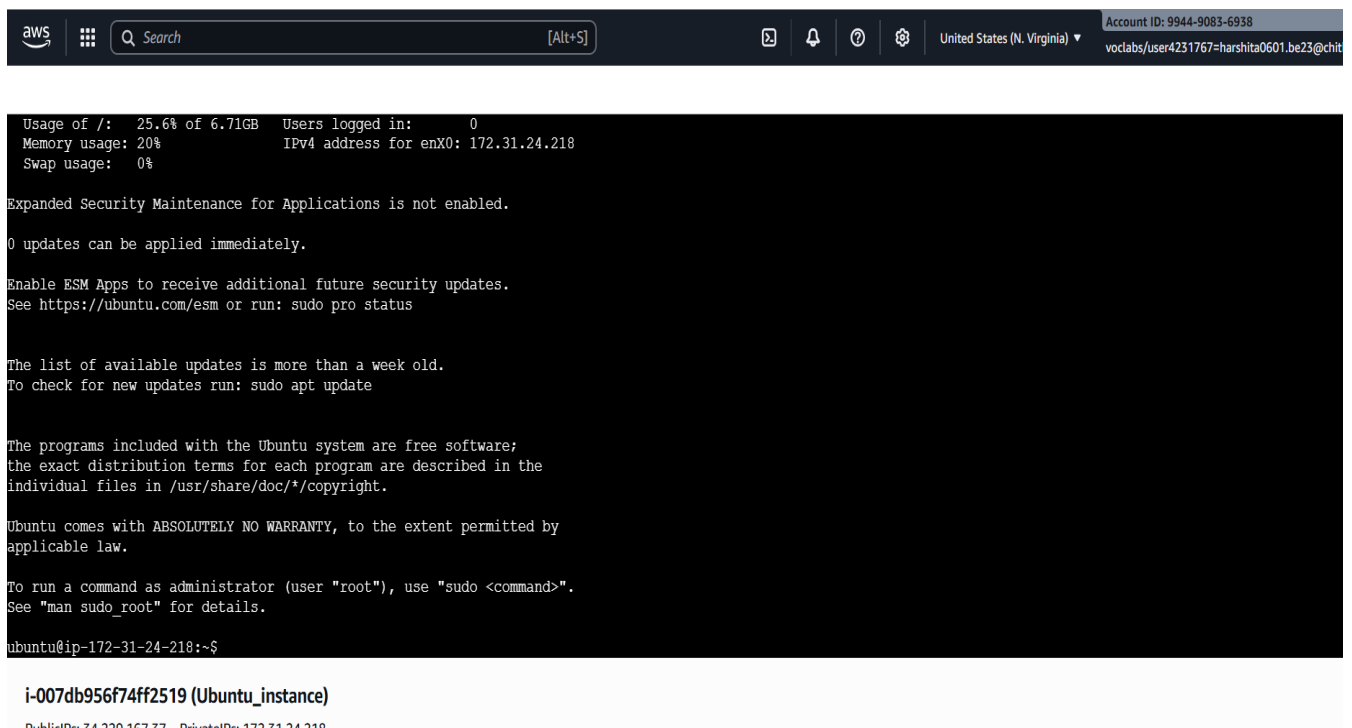


Figure 1.9

```
ubuntu@ip-172-31-22-254: ~
de11@DESKTOP-VH440PV MINGW64 ~
$ cd downloads
de11@DESKTOP-VH440PV MINGW64 ~/downloads
$ chmod 400 ec2.pem
de11@DESKTOP-VH440PV MINGW64 ~/downloads
$ ssh -i ec2.pem ubuntu@ec2-3-87-125-35.compute-1.amazonaws.com
The authenticity of host 'ec2-3-87-125-35.compute-1.amazonaws.com (3.87.125.35)' can't be established.
ED25519 key fingerprint is SHA256:8sS/dCu94soa5hb7U5X0ftPR/MwbKpBM4WnCV5ArX+A.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added 'ec2-3-87-125-35.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1011-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Fri Sep 12 17:42:59 UTC 2025

System load:  0.0           Processes:      106
Usage of /:   25.6% of 6.71GB Users logged in: 0
Memory usage: 20%          IPv4 address for enx0: 172.31.22.254
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-22-254:~$ |
```

Figure 1.10

(B) Windows EC2 Instance

1. Repeat steps 1–3, but select Microsoft Windows Server 2025 Base.

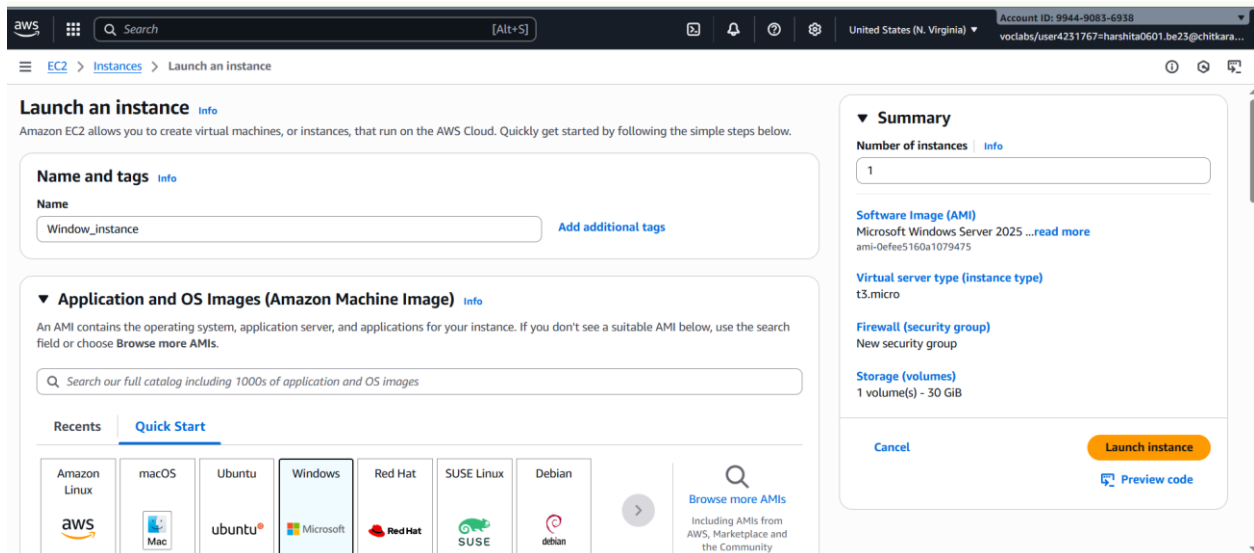


Figure 1.11

2. Allow RDP (Port 3389).
3. Download key file for password decryption.

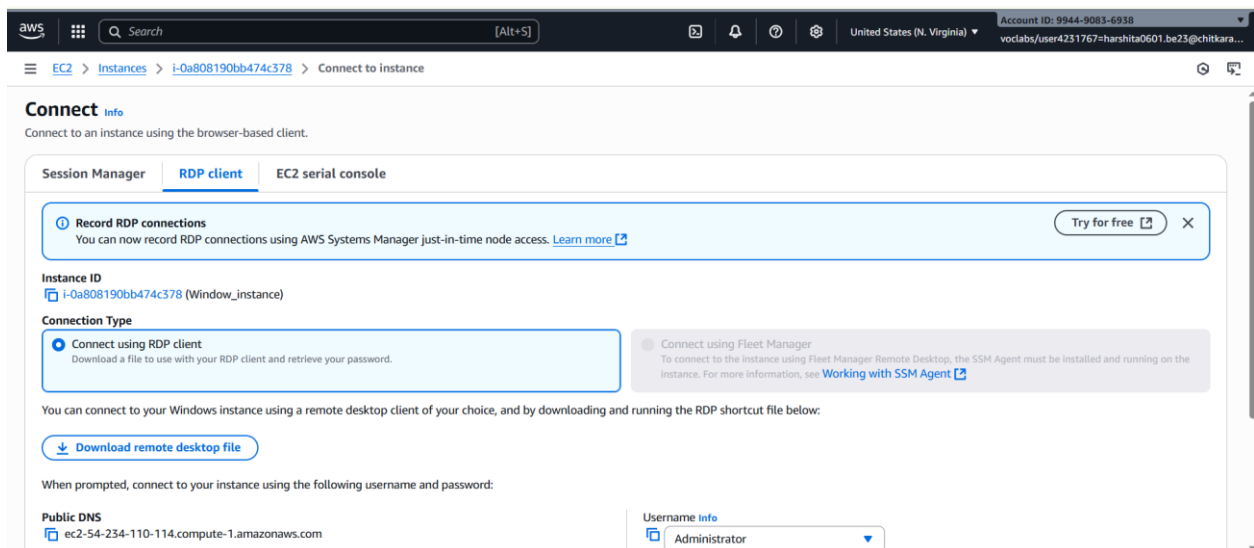


Figure 1.12

4. Launch → Get Password → Decrypt with key file.

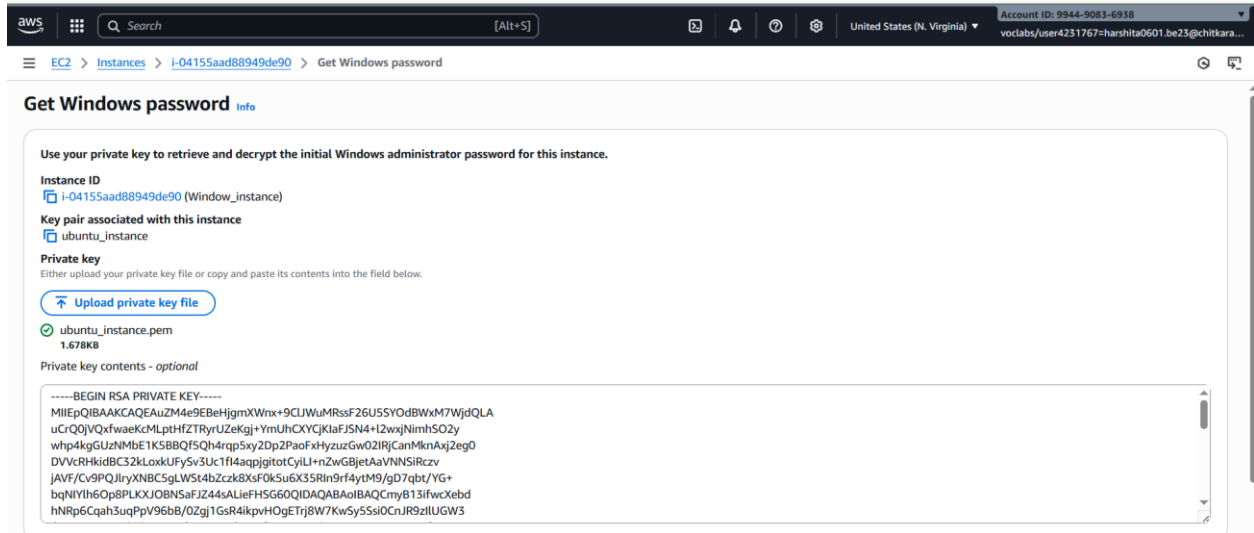


Figure 1.13

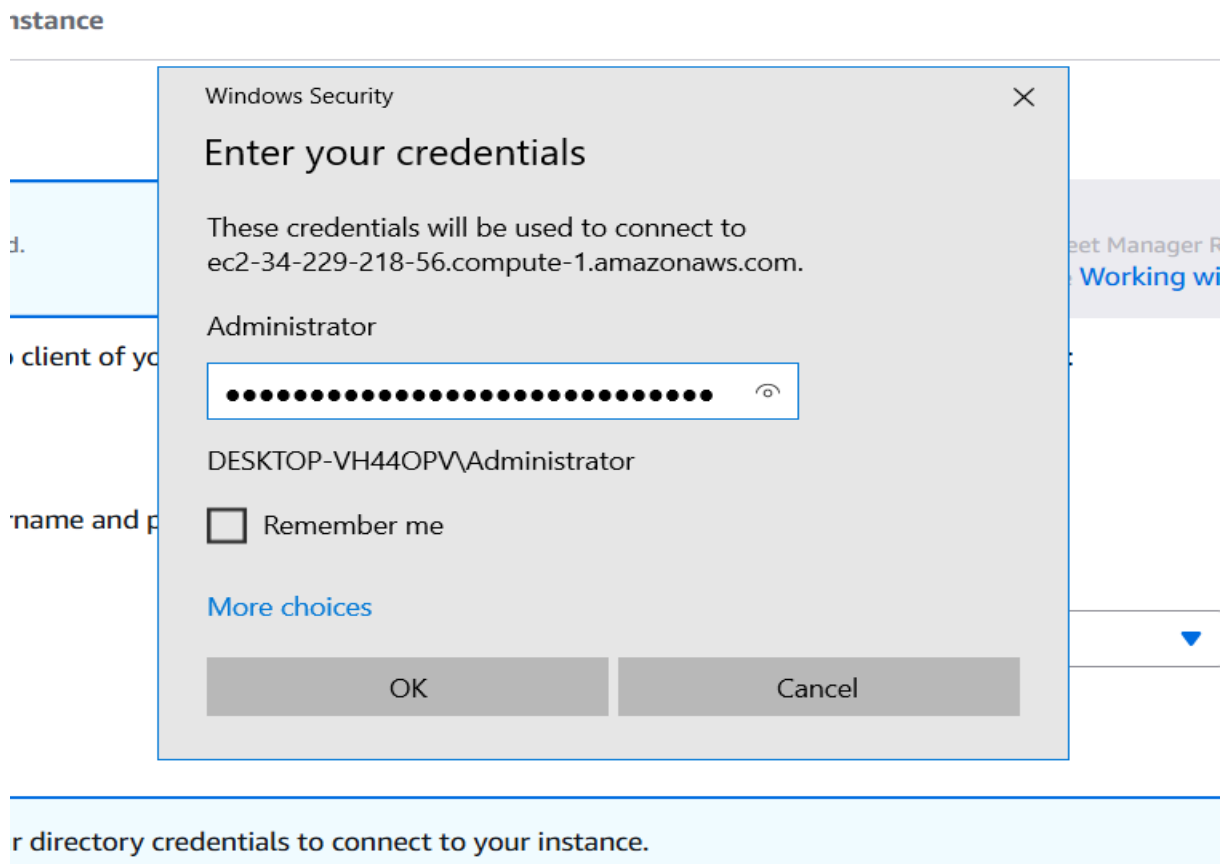


Figure 1.14

5. Connect using Remote Desktop Connection (RDP).



Figure 1.15

Learning Outcomes:

1. Successfully signed up for AWS and explored free-tier services.
2. Launched and connected to both Ubuntu and Windows EC2 instances.
3. Understood basics of creating & managing virtual machines on AWS.

Practical No. 2

Practical Title: Connect to an EC2 Instance via EC2 Connect, SSH Client, and PuTTY.
Modify Security Group Rules.

Objective:

2.1 Connect to an EC2 instance using a key pair file via

- A. EC2 Connect
- B. SSH Client and
- C. PuTTY SSH Client

2.2 Access and modify Security Group inbound and outbound rules to control traffic for EC2 instances.

2.1 Connect to an EC2 instance using a key pair

Step 1: Launch an EC2 Instance

1. Open AWS Management Console → EC2 → Launch Instance.
2. Select Ubuntu 22.04 LTS AMI (Free-Tier Eligible).
3. Choose t2.micro instance type.
4. Create a key pair (e.g., ec2-key.pem) and download it.
5. In Security Group, allow:
 - SSH (Port 22) from your IP.
 - HTTP (Port 80) for web access.

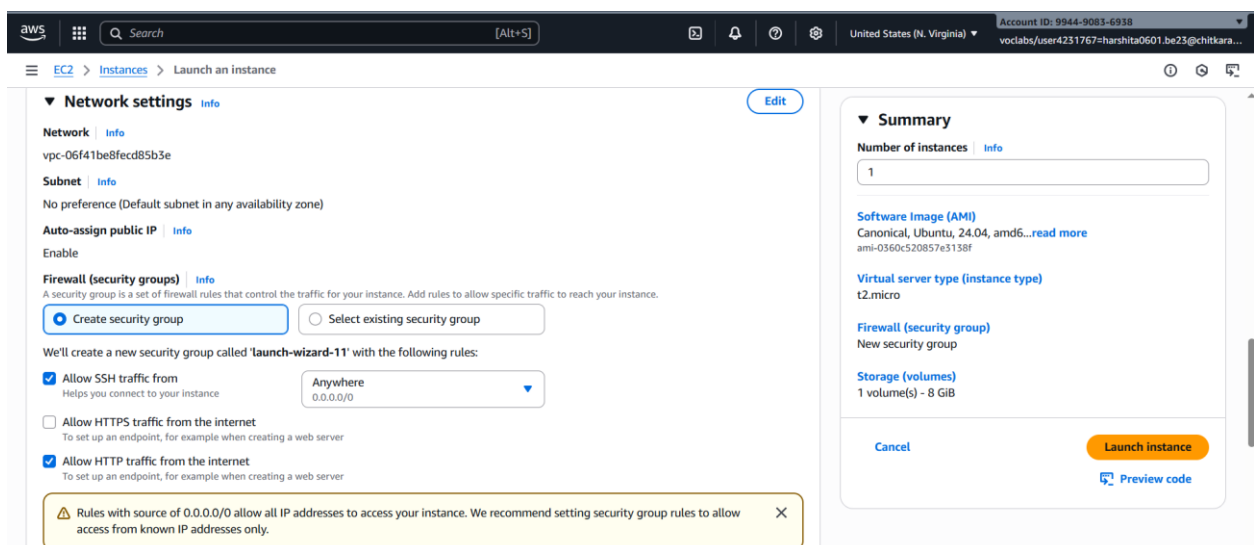


Figure 2.1

Step 2: Connect to EC2 Instance

(A) EC2 Instance Connect (Browser-Based)

1. Go to EC2 → Instances → Select Instance → Connect.
2. Choose EC2 Instance Connect tab.

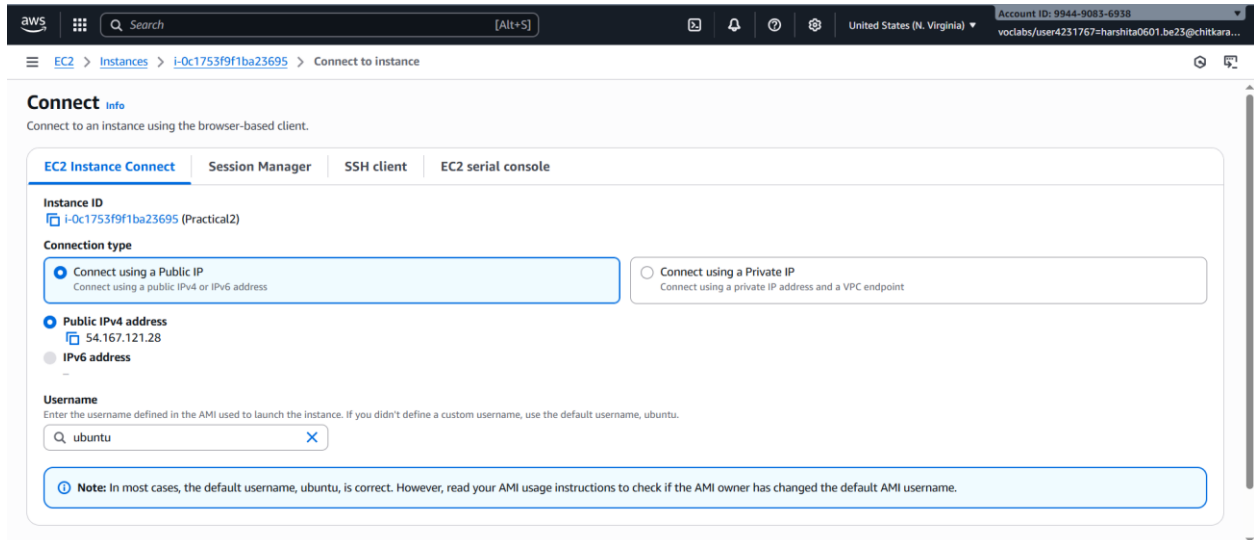


Figure 2.2

3. Click Connect → browser-based terminal opens.

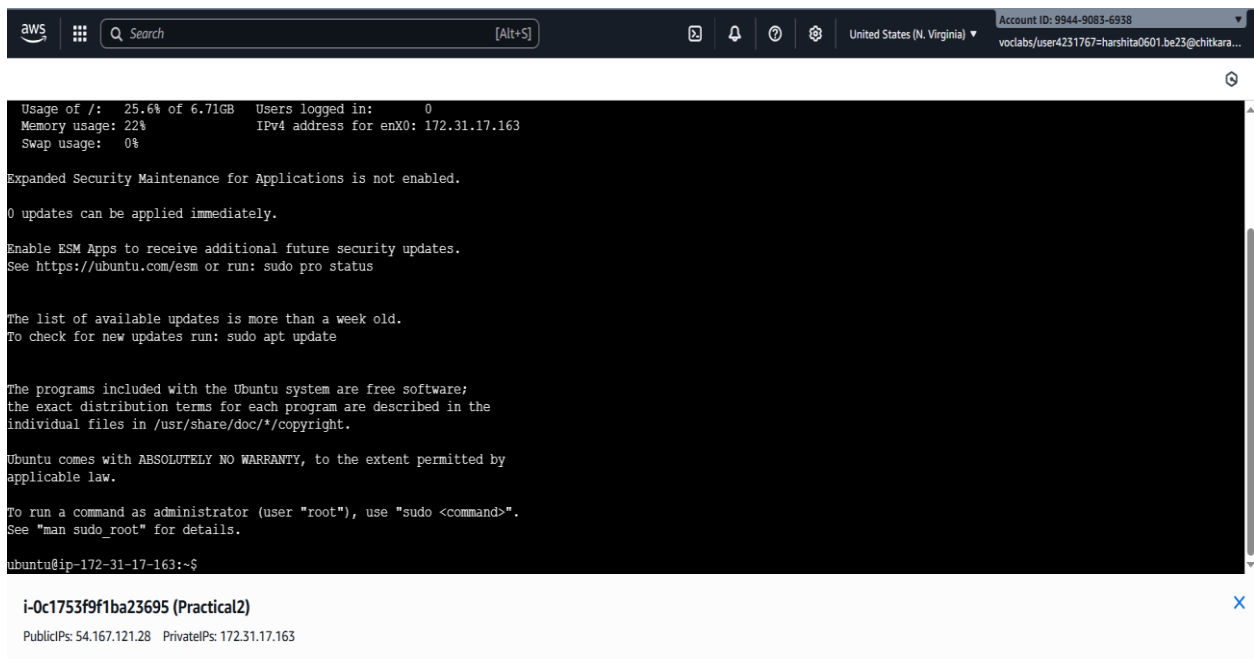


Figure 2.3

(B) SSH Client (Linux/macOS Terminal)

Connect Using SSH Client on Linux/Mac

1. Open Terminal
2. Navigate to the folder where your .pem file is saved: `cd /path/to/your/file`
3. Change Permissions of the Key File

Type this command: `chmod 400 my-key.pem`

4. Connect to your EC2 instance.

Type the SSH command:

`ssh -i my-key.pem ubuntu@ec2<your-public-ip>.compute-1.amazonaws.com`

Replace <your-public-ip> with the Public IPv4 address of your EC2 instance.

```
ubuntu@ip-172-31-22-254: ~
de11@DESKTOP-VH440PV MINGW64 ~
$ cd downloads
de11@DESKTOP-VH440PV MINGW64 ~/downloads
$ chmod 400 ec2.pem
de11@DESKTOP-VH440PV MINGW64 ~/downloads
$ ssh -i ec2.pem ubuntu@ec2-3-87-125-35.compute-1.amazonaws.com
The authenticity of host 'ec2-3-87-125-35.compute-1.amazonaws.com (3.87.125.35)' can't be established.
ED25519 key fingerprint is SHA256:8sS/dCu94soa5hb7U5X0ftPR/MwbKpBM4WnCV5ArX+A.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added 'ec2-3-87-125-35.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1011-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Sep 12 17:42:59 UTC 2025

System load:  0.0               Processes:    106
Usage of /:   25.6% of 6.71GB   Users logged in: 0
Memory usage: 20%              IPv4 address for enX0: 172.31.22.254
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-22-254:~$ |
```

Figure 2.4

(C) PuTTY (Windows)

1. Convert .pem to .ppk using PuTTY gen.
2. Open PuTTY, enter the EC2 Public IP.
3. Under SSH → Auth, load the .ppk key file.
4. Click Open → log in as ubuntu

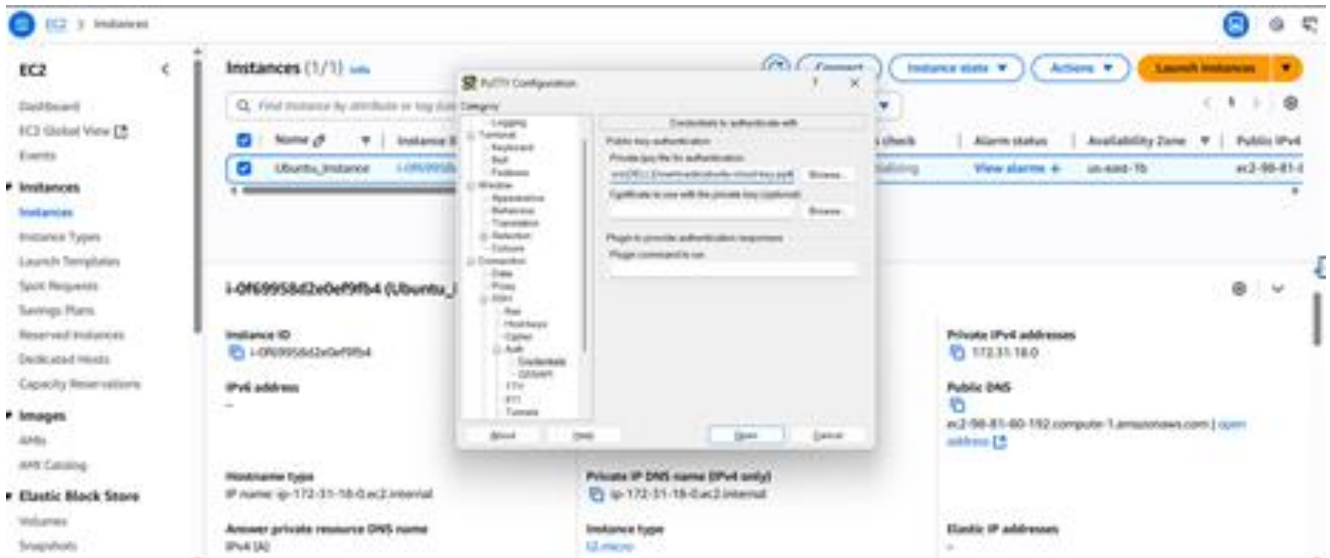


Figure 2.5

```
ubuntu@ip-172-31-18-0: ~
login as: ubuntu
Authenticating with public key "imported-openssh-key"
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1011-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Tue Sep  2 13:28:13 UTC 2025

System load:  0.0               Processes:    106
Usage of /:   25.6% of 6.71GB   Users logged in:  0
Memory usage: 20%              IPv4 address for enX0: 172.31.18.0
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-18-0:~$
```

Figure 2.6

2.2 Access and modify Security Group inbound and outbound rules to control traffic for EC2 instances.

(A) Edit Inbound Rules

1. Go to EC2 → Security Groups → Select SG → Edit inbound rules.
2. Add a rule to allow HTTP (Port 80) from Anywhere (0.0.0.0/0).
3. Remove or restrict SSH access if not needed.

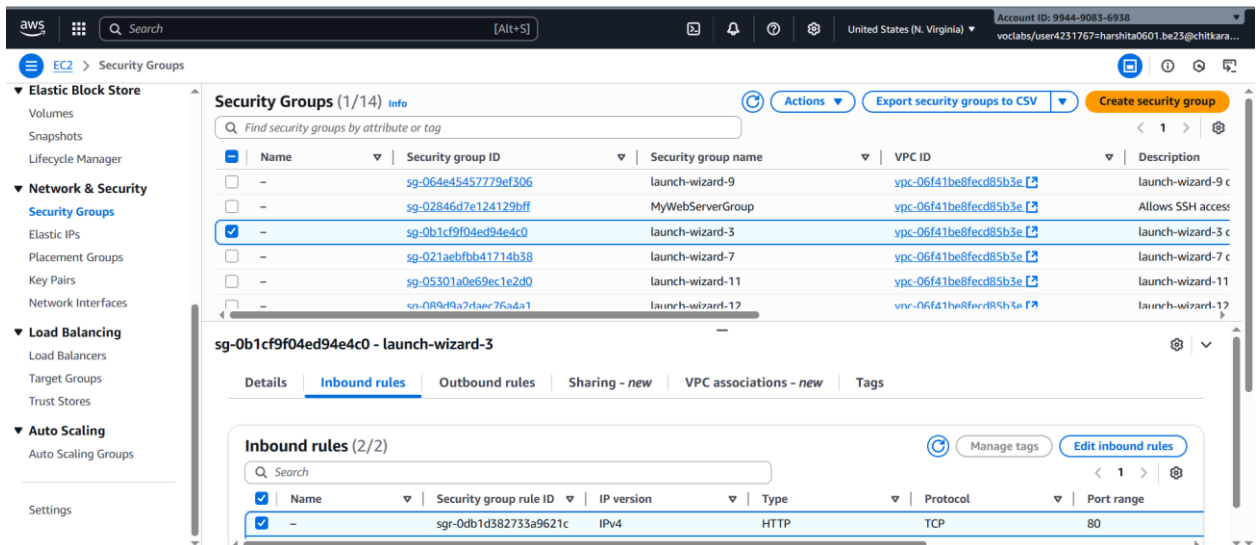


Figure 2.7

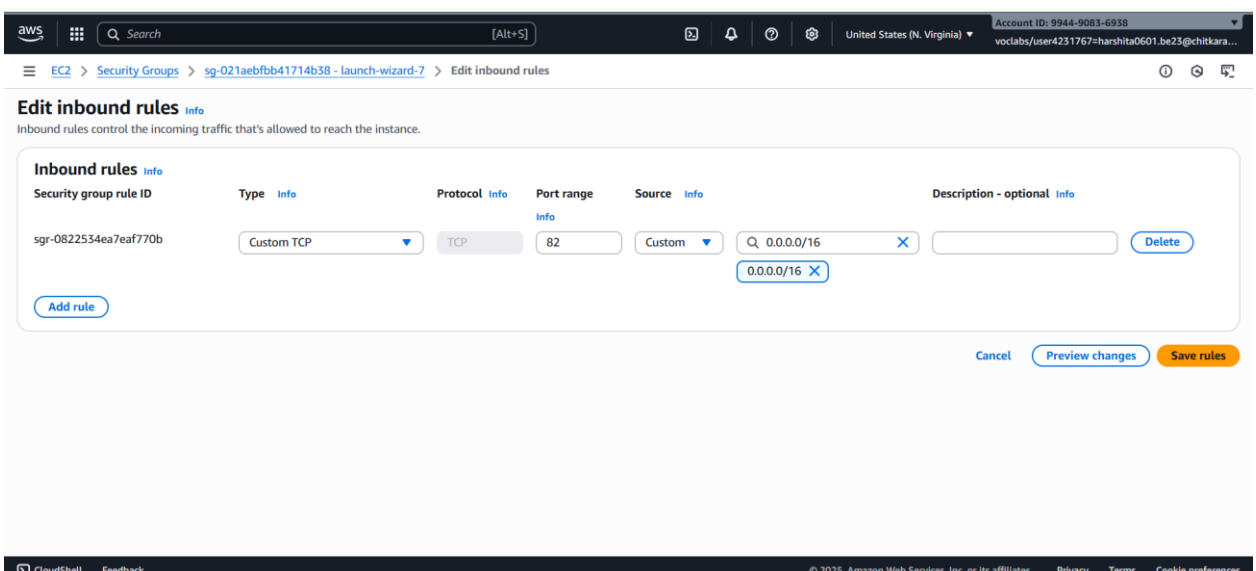


Figure 2.8

(B) Edit Outbound Rules:

1. Go to Outbound rules.
2. Configure allowed outbound traffic (default: all traffic).
3. Save changes.

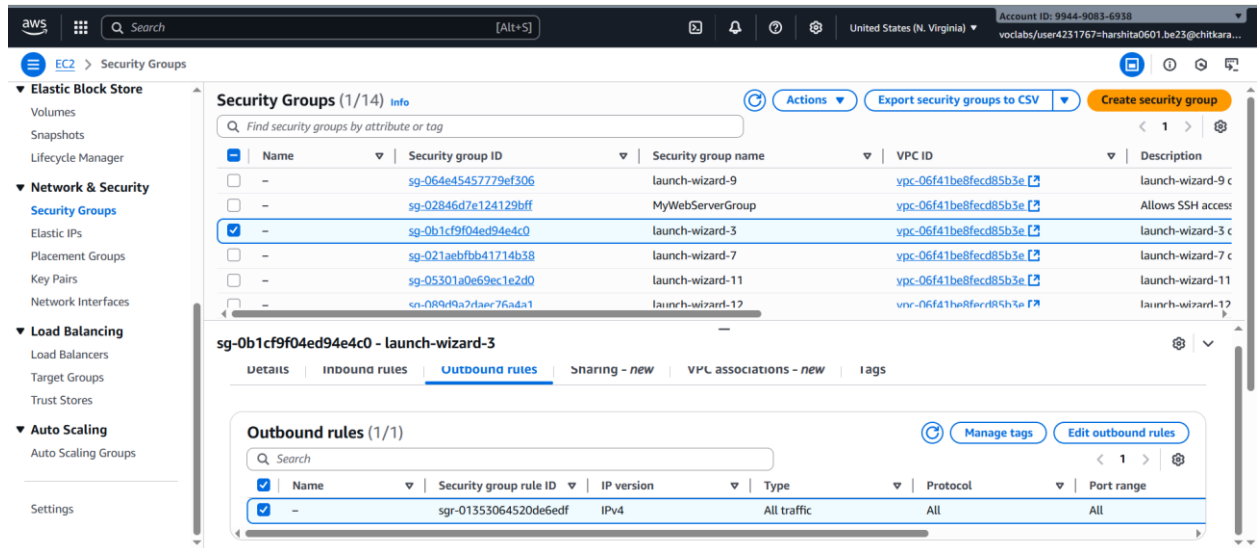


Figure 2.9

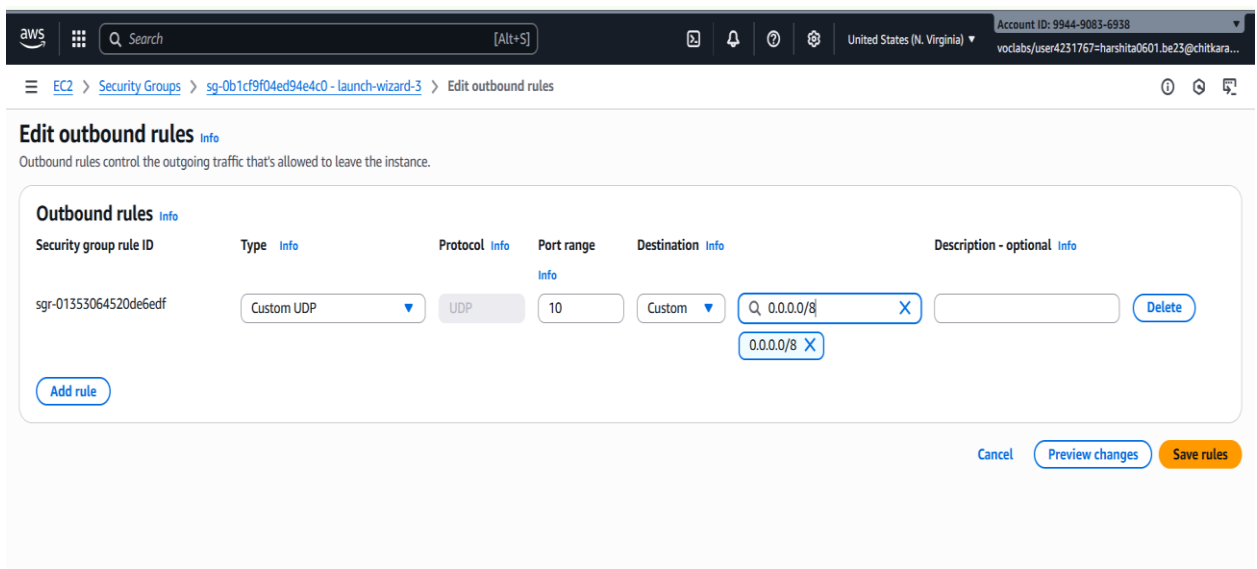


Figure 2.10

Learning Outcomes:

- Successfully connected to EC2 instances using EC2 Instance Connect, SSH Client, AWS CLI, and PuTTY.
- Implemented secure authentication using a key pair file.
- Configured and modified Security Group inbound and outbound rules to control network traffic.
- Gained an understanding of how Security Groups function as a virtual firewall for AWS resource.

Practical No. 3

Practical Title: Set up and configure web servers on EC2 instances, including Apache and Nginxserver. Configure auto-scaling for EC2 instances to handle variable traffic.

Objective:

- A. To install and configure Apache and Nginx web servers on EC2 instances.
- B. To configure Auto Scaling Groups (ASG) for EC2 instances to handle variable traffic automatically.

(A) Install and Configure Apache and Nginx web servers on EC2 instances

Step 1: Launch EC2 Instances

1. Open AWS Management Console → EC2 → Launch Instance
2. Choose Ubuntu 24.04 LTS (HVM)

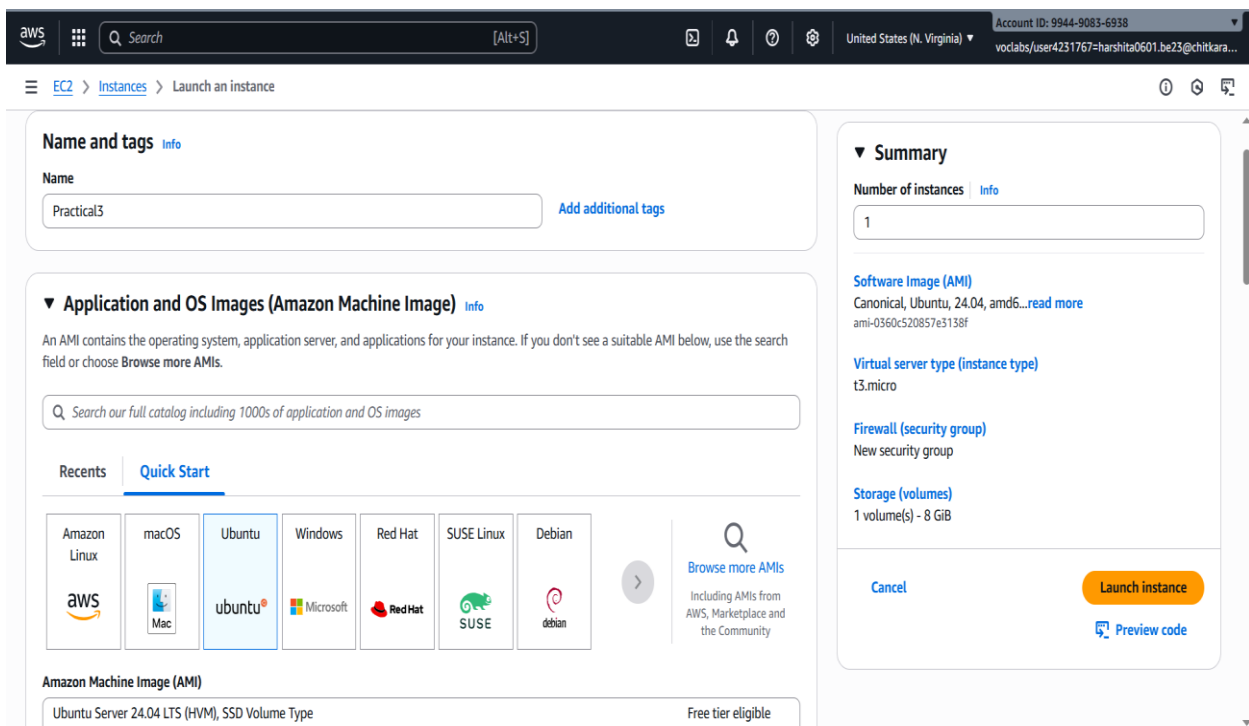


Figure 3.A.1

3. Select t3.micro instance type.
4. Create/Select a key pair (e.g. web key.pem)

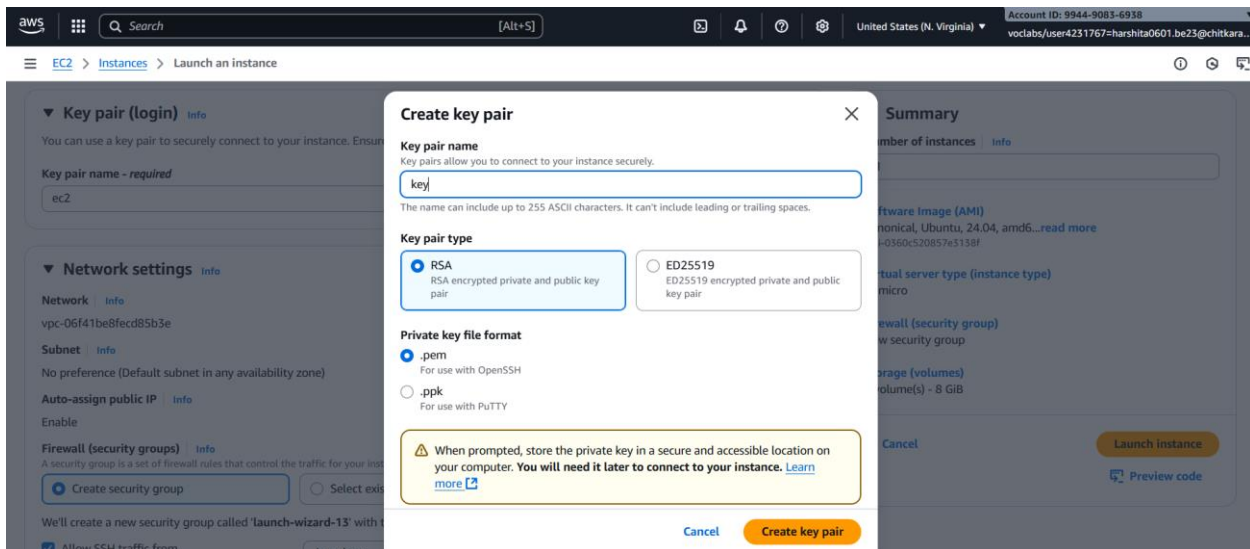


Figure 3.A.2

5. Configure Security Groups to allow:

- HTTP (Port 80).
- HTTPs (Port 443)
- SSH (Port 22)

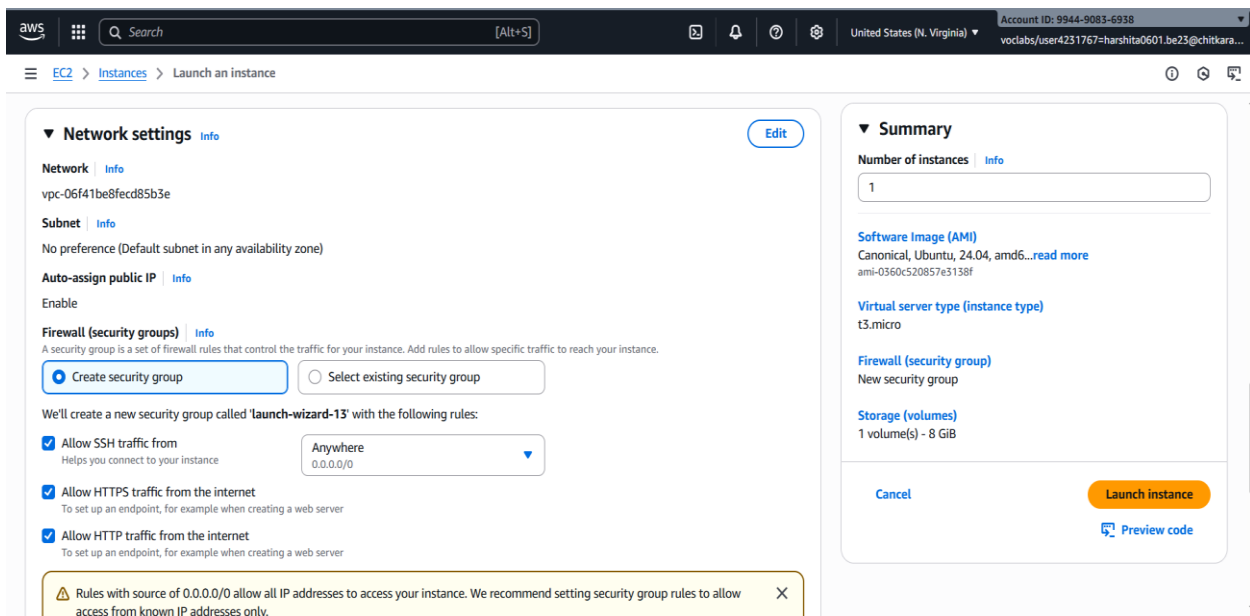


Figure 3.A.3

6. Launch the instance.

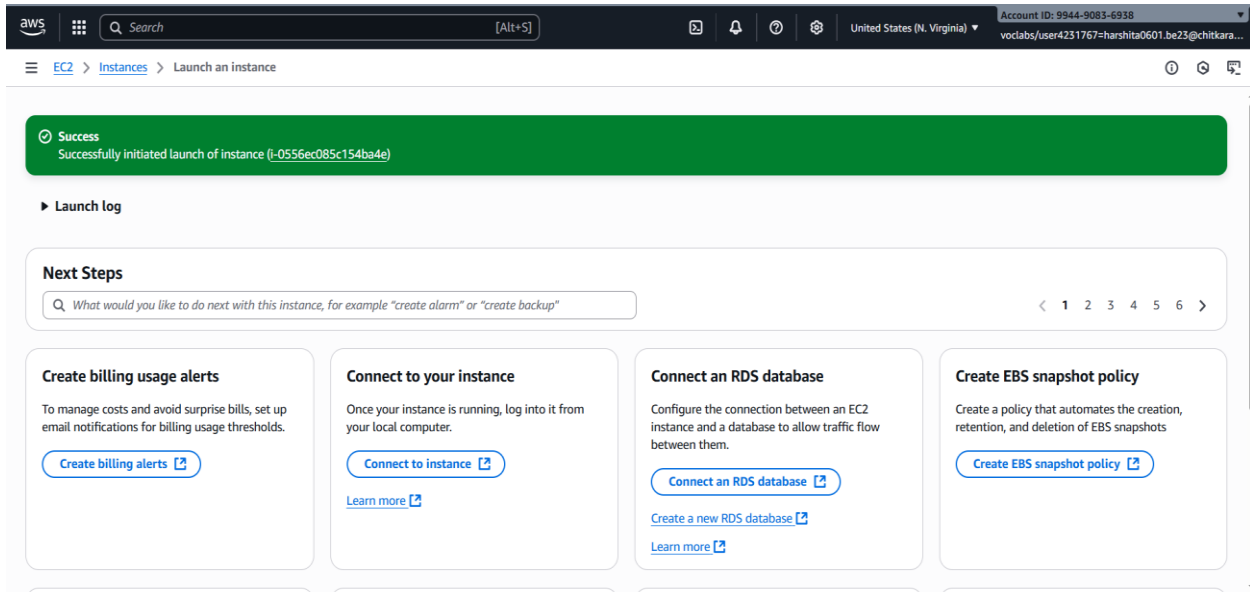


Figure 3.A.4

Step 2: Update and Upgrade the instance

- Run this first to ensure all packages are up to date: `sudo apt update`

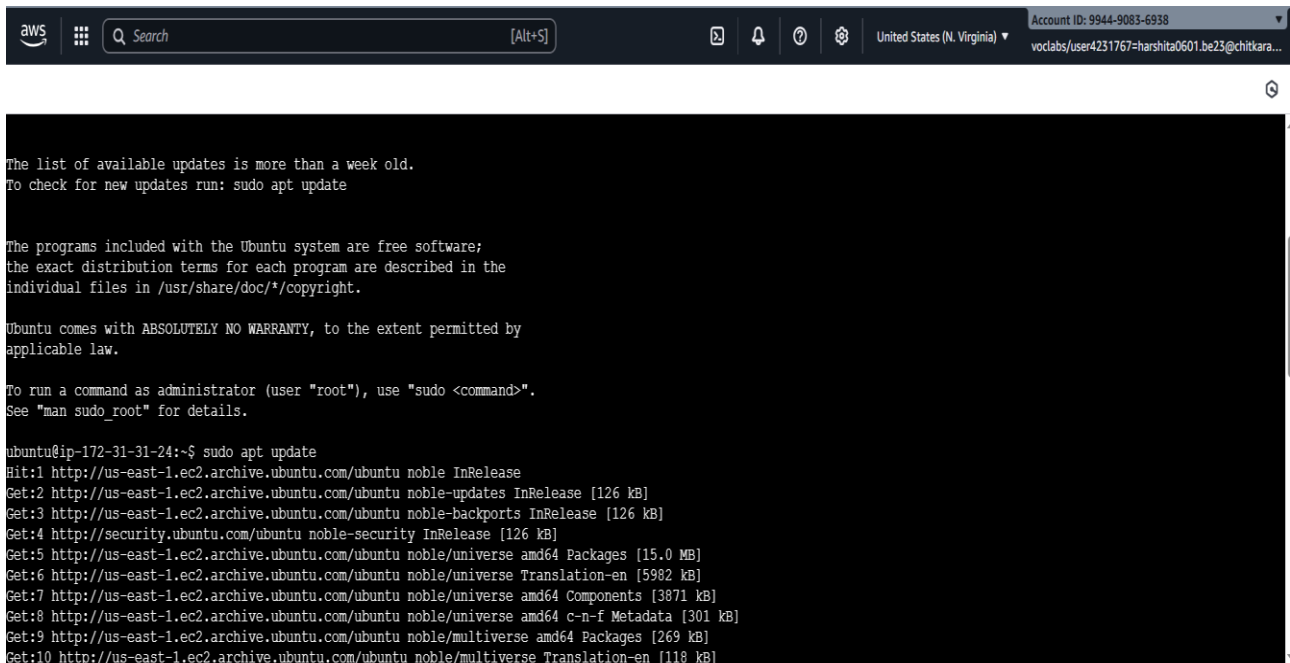


Figure 3.A.5

- Run this first to upgrade the installed packages to their latest versions.: `sudo apt upgrade`

```

Get:53 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [4288 B]
Get:54 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:55 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [380 B]
Fetched 37.1 MB in 6s (6267 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
21 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-31-24:~$ sudo apt upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following NEW packages will be installed:
  linux-aws-6.14-headers-6.14.0-1012 linux-aws-6.14-tools-6.14.0-1012 linux-headers-6.14.0-1012-aws linux-image-6.14.0-1012-aws linux-modules-6.14.0-1012-aws
  linux-tools-6.14.0-1012-aws
The following packages will be upgraded:
  bind9-dnswild bind9-host bind9-libs coreutils fwupd libfwupd2 libpython3.12-minimal libpython3.12-stdlib libpython3.12t64 libudisks2-0 libxml2 linux-aws
  linux-headers-aws linux-image-aws linux-tools-common openssh-client openssh-server openssh-sftp-server python3.12 python3.12-minimal udisks2
21 upgraded, 6 newly installed, 0 to remove and 0 not upgraded.
12 standard LTS security updates
Need to get 92.1 MB of archives.
After this operation, 178 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 coreutils amd64 9.4-3ubuntu6.1 [1413 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 libpython3.12t64 amd64 3.12.3-1ubuntu0.8 [2355 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 python3.12 amd64 3.12.3-1ubuntu0.8 [651 kB]
i.0556er085r154hade (Partial?)

```

Figure 3.A.6

Step 3: Install Apache

- `sudo apt install apache2 -y`

```

No containers need to be restarted.
User sessions running outdated binaries:
ubuntu @ session #1: sshd(1075)

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-31-24:~$ sudo apt install apache2 -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils libapr1t64 libaprutil1-dbd-sqlite3 libaprutil1-ldap libaprutil1t64 liblua5.4-0 ssl-cert
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom www-browser
The following NEW packages will be installed:
  apache2 apache2-bin apache2-data apache2-utils libapr1t64 libaprutil1-dbd-sqlite3 libaprutil1-ldap libaprutil1t64 liblua5.4-0 ssl-cert
0 upgraded, 10 newly installed, 0 to remove and 0 not upgraded.
Need to get 2086 kB of archives.
After this operation, 8090 kB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 libapr1t64 amd64 1.7.2-3.1ubuntu0.1 [108 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libaprutil1t64 amd64 1.6.3-1.1ubuntu7 [91.9 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libaprutil1-dbd-sqlite3 amd64 1.6.3-1.1ubuntu7 [11.2 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libaprutil1-ldap amd64 1.6.3-1.1ubuntu7 [9116 B]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 liblua5.4-0 amd64 5.4.6-3build2 [166 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-bin amd64 2.4.58-1ubuntu8.8 [1331 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-data all 2.4.58-1ubuntu8.8 [163 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-utils amd64 2.4.58-1ubuntu8.8 [97.7 kB]

```

Figure 3.A.7

- `sudo systemctl start apache2` → start Apache immediately
- `sudo systemctl enable apache2` → make it start automatically on boot
- `sudo systemctl status apache2` → verifies apache is working

```

aws
Search [Alt+S]
United States (N. Virginia)
Account ID: 9944-9083-6938
voclabs/user4231767=harshita0601.be23@chitkara...

No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-20-83:~$ sudo systemctl start apache2
ubuntu@ip-172-31-20-83:~$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
ubuntu@ip-172-31-20-83:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-09-12 17:56:15 UTC; 1min 44s ago
     Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 7387 (apache2)
      Tasks: 55 (limit: 1008)
    Memory: 5.3M (peak: 5.4M)
       CPU: 45ms
    CGroup: /system.slice/apache2.service
            └─7387 /usr/sbin/apache2 -k start
              └─7389 /usr/sbin/apache2 -k start
                └─7390 /usr/sbin/apache2 -k start

Sep 12 17:56:15 ip-172-31-20-83 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 12 17:56:15 ip-172-31-20-83 systemd[1]: Started apache2.service - The Apache HTTP Server.
ubuntu@ip-172-31-20-83:~$

i-04969048cccc9880e (Practical3)
PublicIPs: 54.234.30.129 PrivateIPs: 172.31.20.83

```

Figure 3.A.8

Step4: Test in browser:

- Copy your EC2 public IP from the AWS console.
- Open in your browser → `http://<your-public-ip>/`

Ubuntu **It works!**

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is **fully documented in `/usr/share/doc/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

```

/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|-- mods-enabled
|   |-- *.load
|   |-- *.conf
|-- conf-enabled
|   |-- *.conf
|-- sites-enabled
|   |-- *.conf

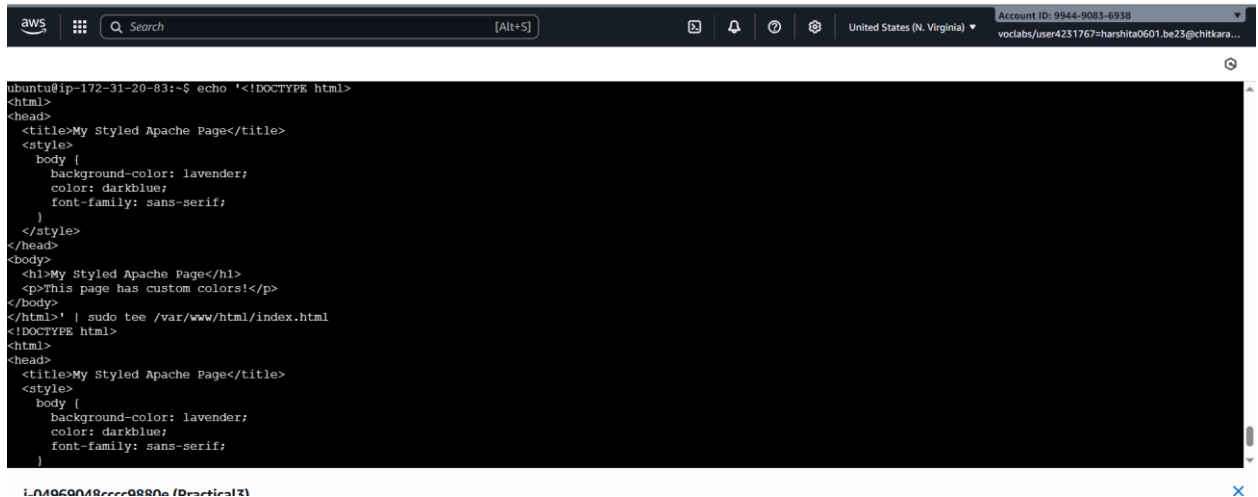
```

- `apache2.conf` is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
- `ports.conf` is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.

Figure 3.A.9

Step 5: Customizing Web Pages using Apache:

- Run this command in the ubuntu terminal
 1. `echo '<body style="background-color:lavender; color:darkblue; font-family:sans-serif;"><h1>My Styled Apache Page</h1><p>This page has custom colors!</p></body>' | sudo tee /var/www/html/index.html`



```

aws
Search [Alt+S]
United States (N. Virginia)
Account ID: 9944-9083-6938
vociabs/user4231767=harshita0601.be23@chitkara...

ubuntu@ip-172-31-20-83:~$ echo '<!DOCTYPE html>
<html>
<head>
<title>My Styled Apache Page</title>
<style>
  body {
    background-color: lavender;
    color: darkblue;
    font-family: sans-serif;
  }
</style>
</head>
<body>
<h1>My Styled Apache Page</h1>
<p>This page has custom colors!</p>
</body>
</html>' | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html>
<head>
<title>My Styled Apache Page</title>
<style>
  body {
    background-color: lavender;
    color: darkblue;
    font-family: sans-serif;
  }
</style>
</head>
<body>
<h1>My Styled Apache Page</h1>
<p>This page has custom colors!</p>
</body>
</html>' | sudo tee /var/www/html/index.html
  
```

Figure 3.A.10

After running this command, refresh your browser page and you will see the changes



Figure 3.A.11

2. `echo "<h1>Welcome to My Apache Server!\</h1>" | sudo tee /var/www/html/index.html`

```

color: darkblue;
font-family: sans-serif;
}
</style>
</head>
<body>
<h1>My Styled Apache Page</h1>
<p>This page has custom colors!</p>
</body>
</html>
ubuntu@ip-172-31-20-83:~$ echo '<h1>My Styled Apache Page</h1><p>This page has custom colors!</p></body>' | sudo tee /var/www/html/index.html

```

Figure 3.A.12

After running this command, refresh your browser page and you will see the changes



Figure 3.A.13

Step 6: For NGINX

1. Stop and disable Apache2 by running the below commands

- `sudo systemctl stop apache2`
- `sudo systemctl disable apache2`

This will free up port 80

```

<h1>My Styled Apache Page</h1><p>This page has custom colors!</p></body>
ubuntu@ip-172-31-20-83:~$ sudo systemctl stop apache2
●: command not found
ubuntu@ip-172-31-20-83:~$ sudo systemctl stop apache2
ubuntu@ip-172-31-20-83:~$ sudo systemctl disable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install disable apache2
Removed "/etc/systemd/system/multi-user.target.wants/apache2.service".
ubuntu@ip-172-31-20-83:~$

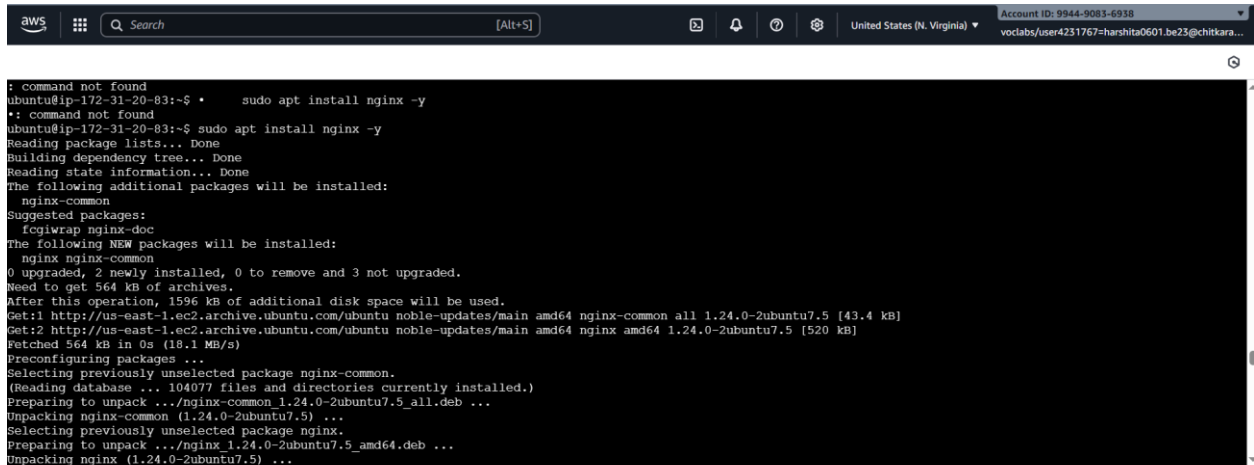
```

Figure 3.A.14

2. Install Nginx (if not already installed)

- `sudo apt update` → refreshes the package list.

- `sudo apt install nginx -y` → installs Nginx and automatically confirms installation.



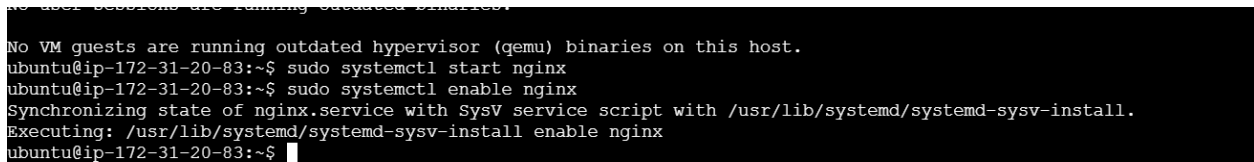
```

command not found
ubuntu@ip-172-31-20-83:~$ sudo apt install nginx -y
*: command not found
ubuntu@ip-172-31-20-83:~$ sudo apt install nginx -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  nginx-common
Suggested packages:
  fcgiwrap nginx-doc
The following NEW packages will be installed:
  nginx nginx-common
0 upgraded, 2 newly installed, 0 to remove and 3 not upgraded.
Need to get 564 KB of archives.
After this operation, 1596 KB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu/noble-updates/main amd64 nginx-common all 1.24.0-2ubuntu7.5 [43.4 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu/noble-updates/main amd64 nginx amd64 1.24.0-2ubuntu7.5 [520 kB]
Fetched 564 kB in 0s (18.1 MB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 104077 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.24.0-2ubuntu7.5_all.deb ...
Unpacking nginx-common (1.24.0-2ubuntu7.5) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.24.0-2ubuntu7.5_amd64.deb ...
Unpacking nginx (1.24.0-2ubuntu7.5) ...

```

Figure 3.A.15

- `sudo systemctl start nginx` → Start Nginx
- `sudo systemctl enable nginx` → Ensures Nginx starts automatically when Ubuntu restarts.



```

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-20-83:~$ sudo systemctl start nginx
ubuntu@ip-172-31-20-83:~$ sudo systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
ubuntu@ip-172-31-20-83:~$

```

i-04969048cccc9880e (Practical3)

Figure 3.A.16

Step 7: Access Nginx in your browser

1. Find your Ubuntu server's public IP → `hostname -I`
2. Open a browser and visit → `http://<Public_IP>`
3. Make sure that nginx is using Port 80 by running the below command
 - `sudo nano /etc/nginx/sites-enabled/default`

```
nginx.conf
listen 80 default_server;
listen [::]:80 default_server;

# SSL config
# include snippets/ssl.conf;

root /var/www/html;

# Add index.php to the list if you are using PHP
index index.php index.html;

server_name _;

location / {
    # First attempt to serve request as file, then
    # as directory, then fall back to displaying a 404.
    try_files $uri $uri/ =404;
}

# pass PHP scripts to FastCGI server
```

Figure 3.A.17

4. After running the above command restart the nginx → `sudo systemctl restart nginx`



Figure 3.A.18

Step 8: Customize Nginx Page

1. `echo "<h1>Welcome to My Nginx Server!</h1>" | sudo tee /usr/share/nginx/html/index.html`

Welcome to My Nginx Server!

Figure 3.A.19

2. Create a Full HTML Page with Form and Style

```
<!DOCTYPE html>
<html>
<head>
  <title>My Nginx Webpage</title>
  <style>
    body {
      background-color: #fdf6e3;
      font-family: Arial, sans-serif;
      text-align: center;
      padding: 50px;
    }
    input, textarea {
      padding: 10px;
      margin: 10px;
      width: 250px;
    }
    button {
      padding: 10px 20px;
      font-size: 16px;
    }
  </style>
</head>
<body>
  <h1>Welcome to My Nginx Page!</h1>
  <p>Please enter your name and message:</p>
  <form onsubmit="alert('Thank you for submitting!'); return false;">
    <input type="text" placeholder="Your Name" required><br>
    <textarea placeholder="Your Message" required></textarea><br>
    <button type="submit">Submit</button>
  </form>
  <p style="margin-top:40px;">This page is powered by Nginx on EC2!</p>
</body>
</html>
```

Figure 3.A.20

Welcome to My Nginx Page!

Please enter your name and message:

Your Name

Your Message

Submit

This page is powered by Nginx on EC2!

Welcome to My Nginx Server!

Figure 3.A.21

(B) Configure auto-scaling for EC2 instances to handle variable traffic.

Step 1: Create a Launch Template

1. Go to EC2 Console → Launch Template → Click Create Launch Template.

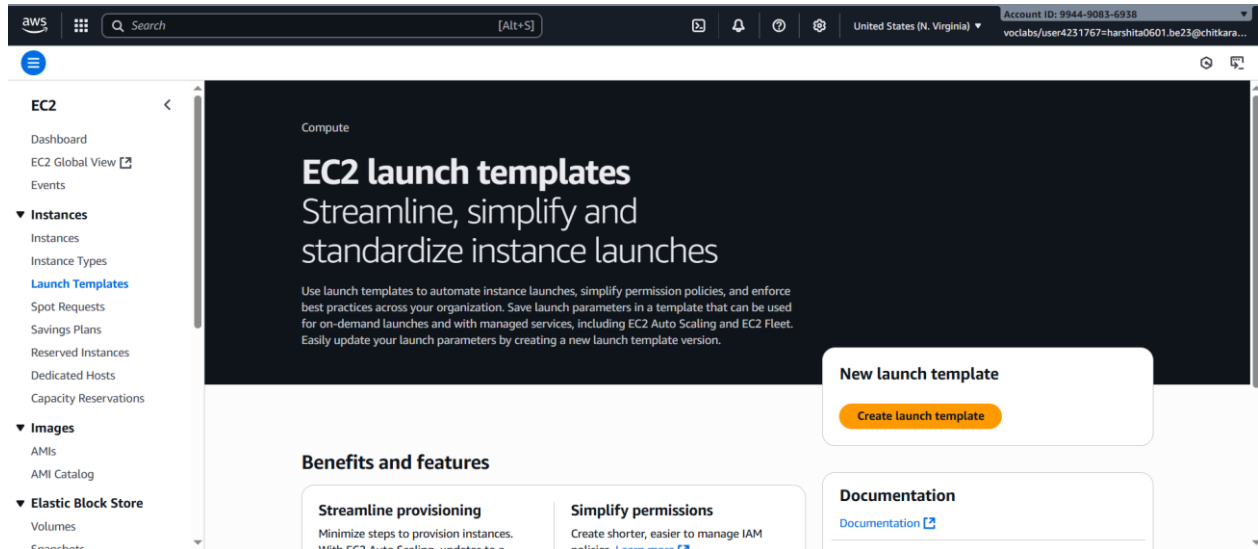


Figure 3.B.1

2. Fill in the basic information.

- Name : eg:- web-server-template
- Instance Type:- eg:- t2.micro or t3.micro
- Key Pair:- Select your existing key.
- Security Group:- Ensure it allows HTTP (port 80) or 8080.
- User Data:- (Optional) Add shell script to auto-install web server if not in AMI.

Create launch template

Creating a launch template allows you to create a saved instance configuration that can be reused, shared and launched at a later time. Templates can have multiple versions.

Launch template name and description

Launch template name - *required*

web-server-template

Must be unique to this account. Max 128 chars. No spaces or special characters like '&', '!', '@'.

Template version description

Initial Version

Max 255 chars

Auto Scaling guidance [Info](#)

Select this if you intend to use this template with EC2 Auto Scaling

☐ Provide guidance to help me set up a template that I can use with EC2 Auto Scaling

► **Template tags**

► **Source template**

Summary

Software Image (AMI)

-

Virtual server type (instance type)

-

Firewall (security group)

-

Storage (volumes)

-

[Cancel](#) [Create launch template](#)

Figure 3.B.2

3. Click Create Launch Template.

vp-06f41be8fcd85b3e
172.31.0.0/16

Inbound Security Group Rules

▼ Security group rule 1 (TCP, 80, 0.0.0.0/0, Allow web Access)

[Remove](#)

Type [Info](#) **Protocol** [Info](#) **Port range** [Info](#)

HTTP TCP 80

Source type [Info](#) **Source** [Info](#) **Description - optional** [Info](#)

Custom [Add CIDR, prefix list or security group](#) Allow web Access

0.0.0.0/0 [X](#)

▼ Security group rule 2 (TCP, 22, 0.0.0.0/0, Allow ssh for management)

[Remove](#)

Type [Info](#) **Protocol** [Info](#) **Port range** [Info](#)

ssh TCP 22

Source type [Info](#) **Source** [Info](#) **Description - optional** [Info](#)

Custom [Add CIDR, prefix list or security group](#) Allow ssh for management

0.0.0.0/0 [X](#)

Summary

Software Image (AMI)

Amazon Linux 2023 AMI 2023.8.2...[read more](#)

ami-0150ccaf51ab55a51

Virtual server type (instance type)

-

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

[Cancel](#) [Create launch template](#)

Figure 3.B.3

4. Successfully created

Success

Successfully created Practical3-template(it-0a20a261790c8b4eb).

► **Actions log**

Next Steps

Launch an instance

With On-Demand Instances, you pay for compute capacity by the second (for Linux, with a minimum of 60 seconds) or by the hour (for all other operating systems) with no long-term commitments or upfront payments. Launch an On-Demand Instance from your launch template.

[Launch instance from this template](#)

Create an Auto Scaling group from your template

Amazon EC2 Auto Scaling helps you maintain application availability and allows you to scale your Amazon EC2 capacity up or down automatically according to conditions you define. You can use Auto Scaling to help ensure that you are running your desired number of Amazon EC2 instances during demand spikes to maintain performance and decrease capacity during lulls to reduce costs.

[Create Auto Scaling group](#)

Create Spot Fleet

A Spot Instance is an unused EC2 instance that is available for less than the On-Demand price. Because Spot Instances enable you to request unused EC2 instances at steep discounts, you can lower your Amazon EC2 costs significantly. The hourly price for a Spot Instance (of each instance type in each Availability Zone) is set by Amazon EC2, and adjusted gradually based on the long-term supply of and demand for Spot Instances. Spot instances are well-suited for data-analysis, batch jobs, background processing, and optional tasks.

[Create Spot Fleet](#)

Figure 3.B.4

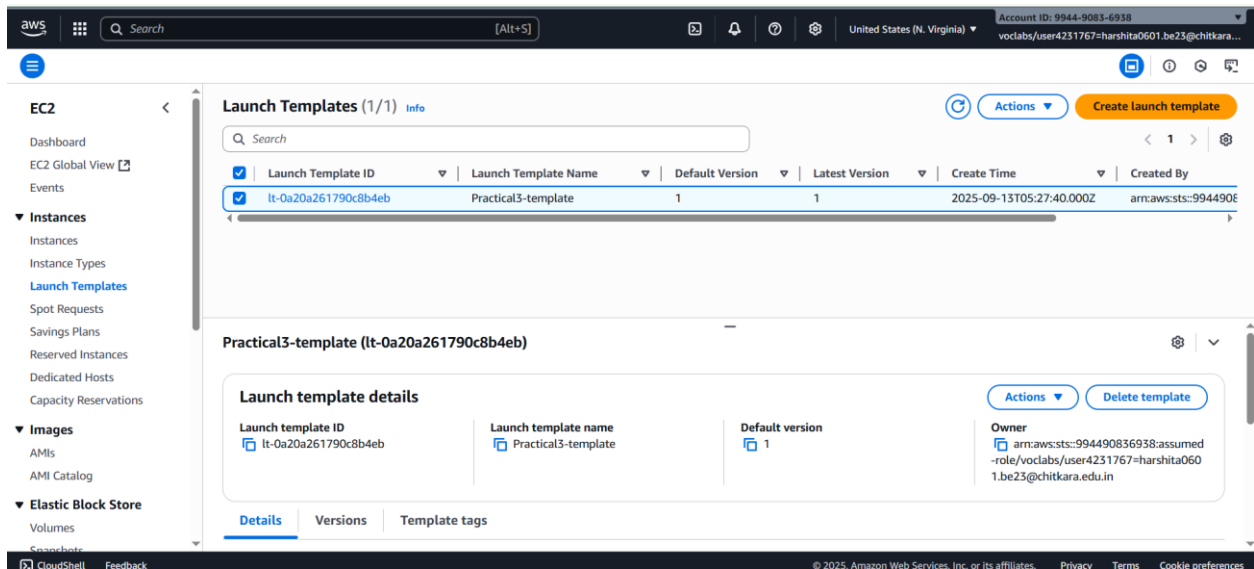


Figure 3.B.5

Step 2: Create an Auto Scaling Group (ASG)

1. Go to EC2 Console → Auto Scaling Groups → Click Create Auto Scaling Groups.

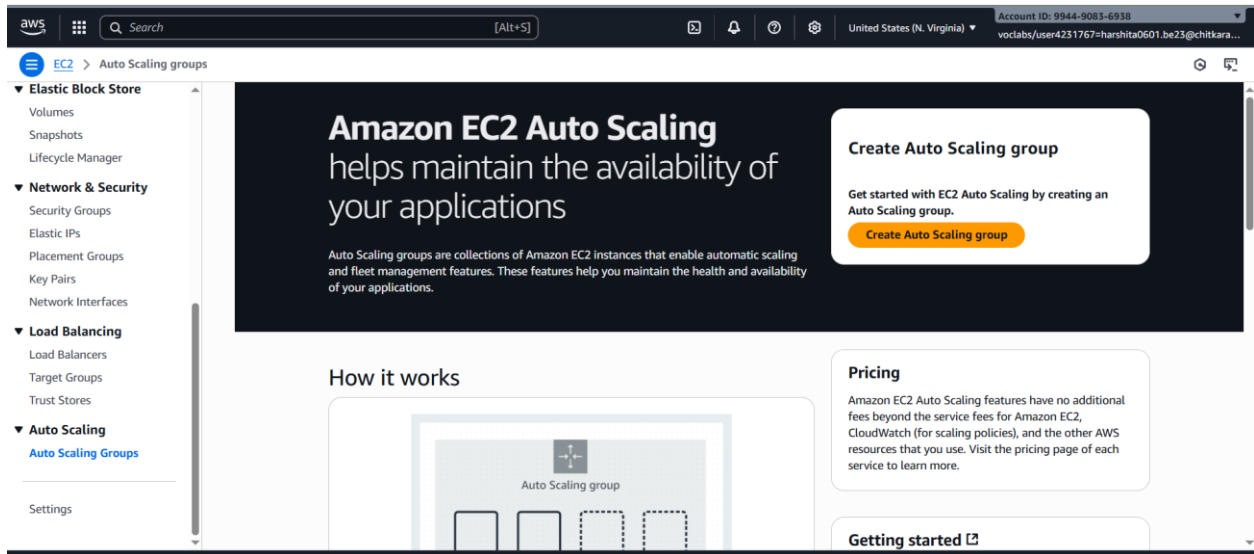


Figure 3.B.6

2. **Name:** e.g., web-auto-scaling
3. **Choose launch template:** Select the one you just created.
4. Click **Next**.

The screenshot shows the 'Choose launch template' step in the AWS 'Create Auto Scaling group' wizard. On the left, a progress bar lists steps from 'Choose launch template' to 'Review'. The main content area has a 'Name' field with 'web-auto-scaling' entered. Below it, a 'Launch template' section includes a dropdown menu set to 'Template' and a 'Version' dropdown set to 'Default (1)'. A blue information box states: 'For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling groups with launch configurations is not recommended but still available via the CLI and API until December 31, 2023.'

Figure 3.B.7

Step 3: Configure Network and Subnet

1. Select a **VPC** and **at least 2 subnets in different Availability Zones** (for high availability).
2. Enable **public IP** assignment if needed.

The screenshot shows the 'Network' step in the AWS 'Create Auto Scaling group' wizard. It features a 'VPC' dropdown menu with 'vpc-06f41be8ecd85b3e' selected. Below, the 'Availability Zones and subnets' section shows two selected subnets: 'use1-az1 (us-east-1a) | subnet-00caaa9022e1bce5' and 'use1-az2 (us-east-1b) | subnet-033767519afb8861c'. At the bottom, the 'Availability Zone distribution' section has 'Balanced best effort' selected, with a description: 'If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.'

Figure 3.B.8

Step 4: Configure Load Balancer

1. Choose **Attach to a new load balancer** (Application Load Balancer).
2. Set:
 - Load balancer name
 - **Scheme:** Internet-facing
 - **Listeners:** HTTP port 80

The screenshot shows the AWS Management Console interface for creating an Auto Scaling group. The navigation pane on the left indicates the current step is 'Step 3 - optional: Integrate with other services'. The main content area is titled 'Load balancing' and provides instructions on how to attach the Auto Scaling group to a load balancer. Three options are presented: 'No load balancer', 'Attach to an existing load balancer', and 'Attach to a new load balancer'. The 'Attach to a new load balancer' option is selected. Under this option, the 'Load balancer type' is set to 'Application Load Balancer' (HTTP, HTTPS). The 'Load balancer name' is 'web-auto-scaling-1', and the 'Load balancer scheme' is 'Internet-facing'.

Figure 3.B.9

3. Add at least **2 Availability Zones**.
4. Create or select a **Target Group**, protocol: HTTP, port 80 (or 8080 if you're using it).

The screenshot shows the 'Network mapping' section of the 'Create Auto Scaling group' page. It indicates that the new load balancer will be created using the same VPC and Availability Zone selections as the Auto Scaling group. The VPC is 'vpc-06f41be8fcd85b3e'. Under 'Availability Zones and subnets', several zones are selected, each with a corresponding subnet: 'use1-az1 (us-east-1a)' with 'subnet-00caaa9022e1bce5', 'use1-az2 (us-east-1b)' with 'subnet-033767519afb8861c', 'use1-az6 (us-east-1d)' with 'subnet-01a2176fca8acd001', 'use1-az3 (us-east-1e)' with 'subnet-0611b38068f4f1284', 'use1-az5 (us-east-1f)' with 'subnet-0b4db9cfa8b07e55c', and 'use1-az4 (us-east-1c)' with 'subnet-067aa5e46f4ce6757'.

Figure 3.B.10

Step 5: Configure Scaling Policies

1. **Desired capacity:** 2 (minimum recommended)
2. Minimum capacity: 1
3. **Maximum capacity:** 3 (or higher based on expected traffic)

The screenshot shows the AWS Management Console interface for creating an Auto Scaling group. The left sidebar lists steps from 2 to 7, with Step 4, 'Configure group size and scaling', highlighted. The main content area is titled 'Define your group's desired capacity and scaling limits. You can optionally add automatic scaling to adjust the size of your group.' It contains three sections: 'Group size' with a 'Desired capacity type' dropdown set to 'Units (number of instances)' and a 'Desired capacity' input field set to '2'; 'Scaling' with 'Scaling limits' input fields for 'Min desired capacity' (2) and 'Max desired capacity' (3); and 'Automatic scaling - optional' with a link to 'Choose whether to use a target tracking policy'.

Figure 3.B.11

4. Choose **Target tracking scaling policy**:

- Metric type: **Average CPU utilization**
- Target value: e.g., **60%**
- This means: add or remove instances to keep average CPU at 60%.

The screenshot shows the 'Review and Create' step of the AWS Auto Scaling group creation process. It displays the 'Target value' set to '60' and 'Instance warmup' set to '300 seconds'. Under 'Instance maintenance policy', there are four options: 'No policy' (selected), 'Prioritize availability', 'Control costs', and 'Flexible'. Each option has a brief description of its behavior during instance replacement events.

Figure 3.B.12

Step 6: Review and Create

- Review all configuration.
- Click **Create Auto Scaling group**.

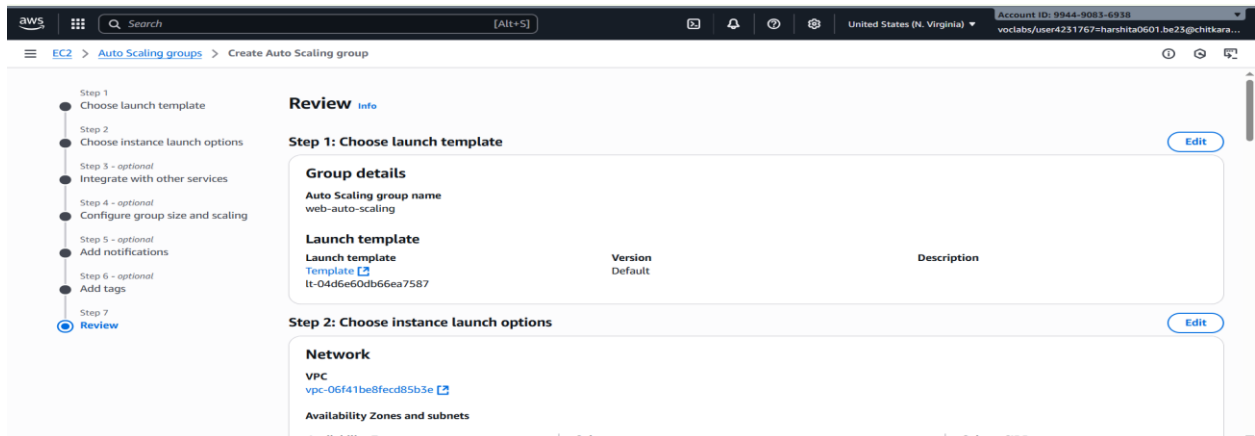


Figure 3.B.13

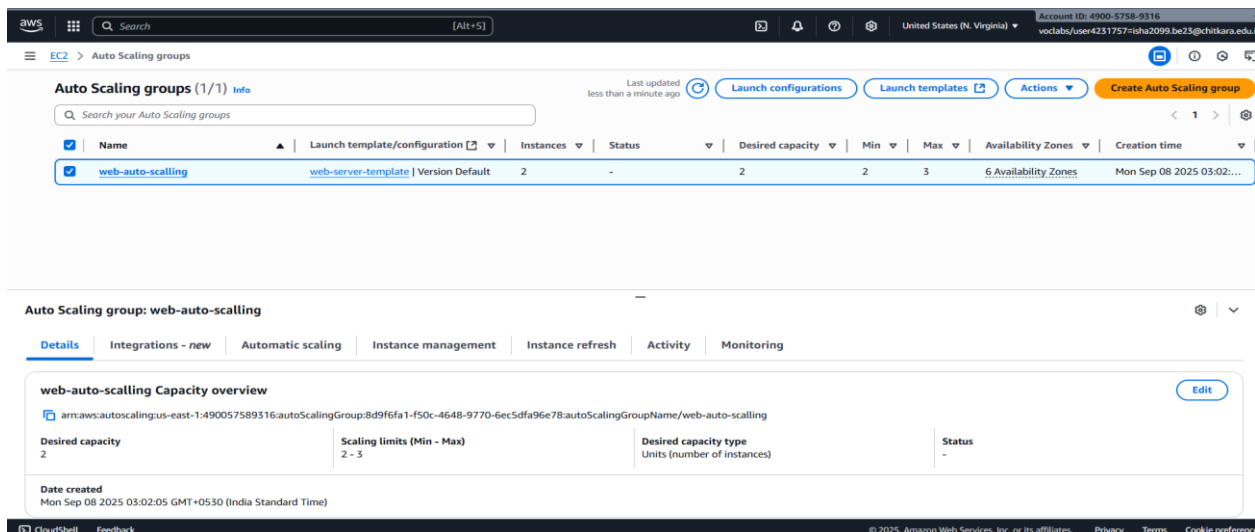


Figure 3.B.14

Step 7: Check the instances

You will see at least two instances created with the help of Auto Scaling.

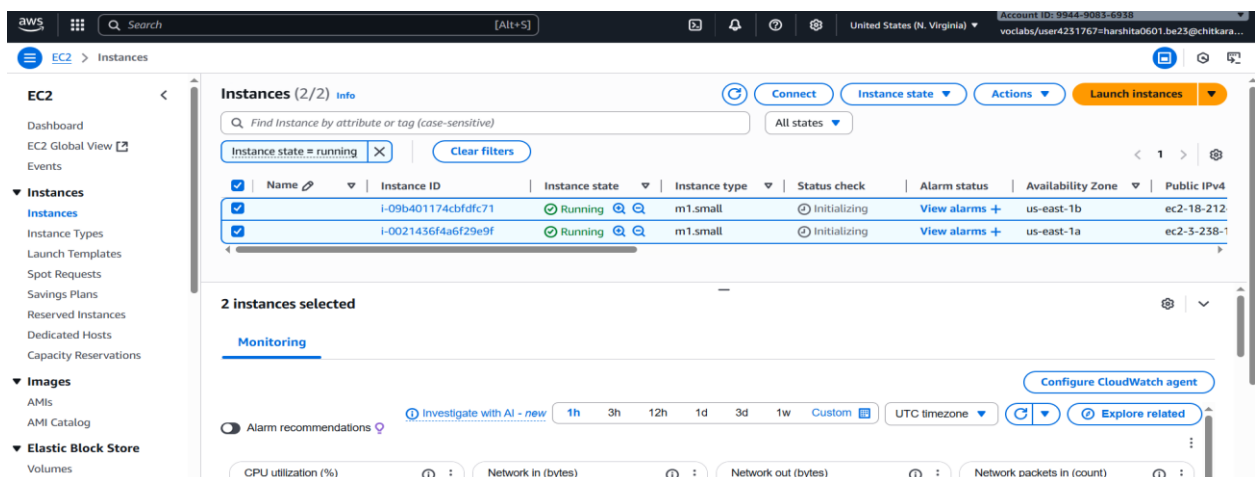


Figure 3.B.15

Practical No. 4

Practical Title: Create Amazon S3 Bucket, Upload Objects, Configure Access, and Host Static Website.

Objective:

1. To create an Amazon S3 bucket for storing data.
2. To upload objects (files such as HTML, CSS, images) into the S3 bucket.
3. To configure access controls using bucket policies for secure and controlled access.
4. To enable static website hosting on the S3 bucket.

Step-by-Step Procedure with Screenshots:

Step1: Create S3 Bucket

1. Sign in to your AWS Management Console → Go to S3 service.

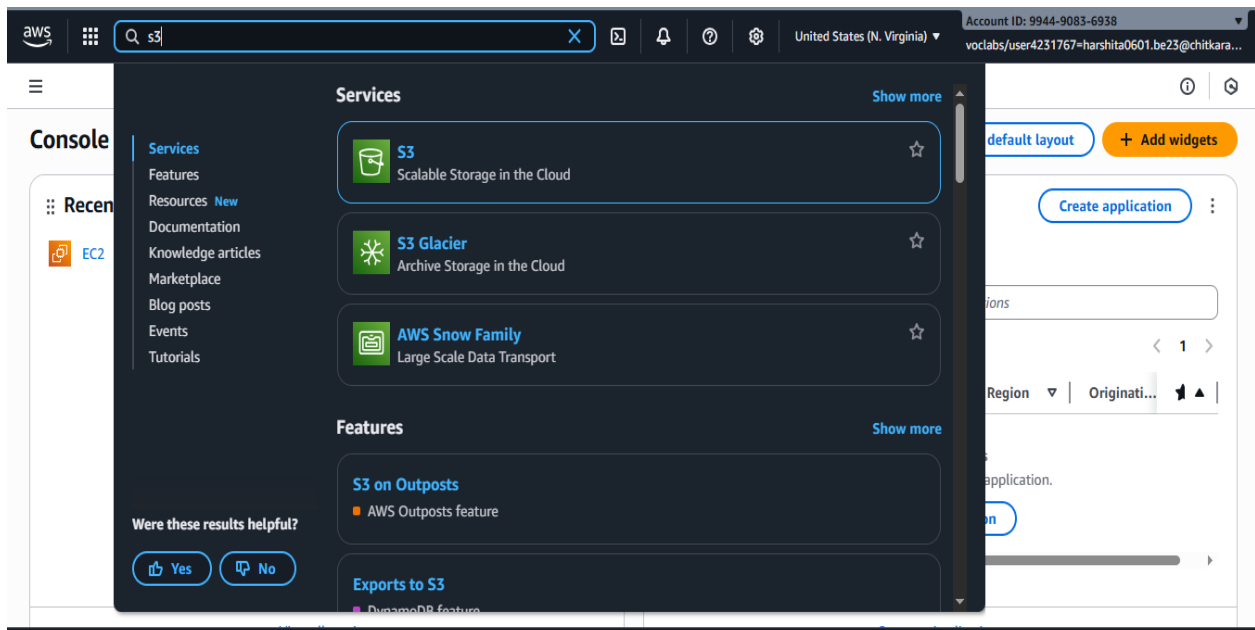


Figure 4.1

- Click on Create bucket.

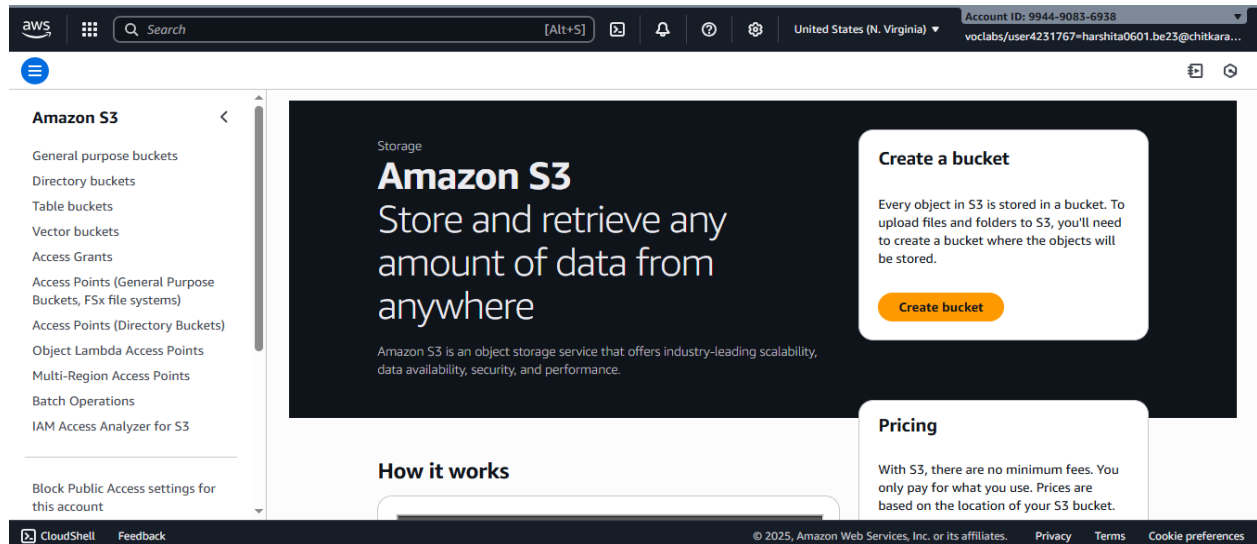


Figure 4.2

- Enter a **unique bucket name** (e.g., my-static-site-bucket).

- Bucket names must be globally unique.

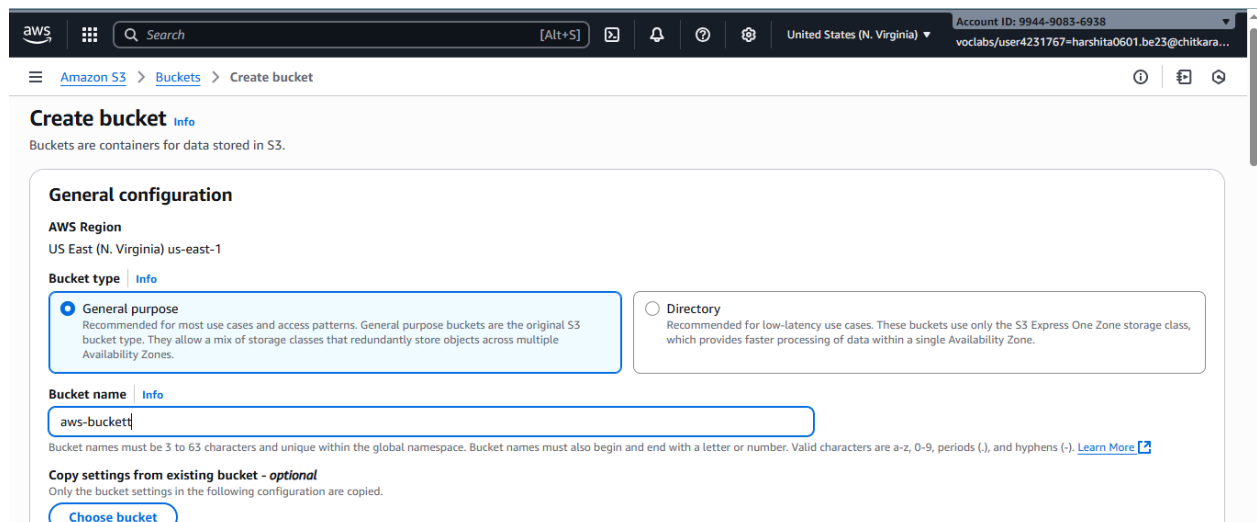


Figure 4.3

- Select an **AWS Region** (pick the one closest to your users).
- In **Block Public Access settings**, uncheck “Block all public access” → Confirm the warning (since you need the bucket public for hosting).

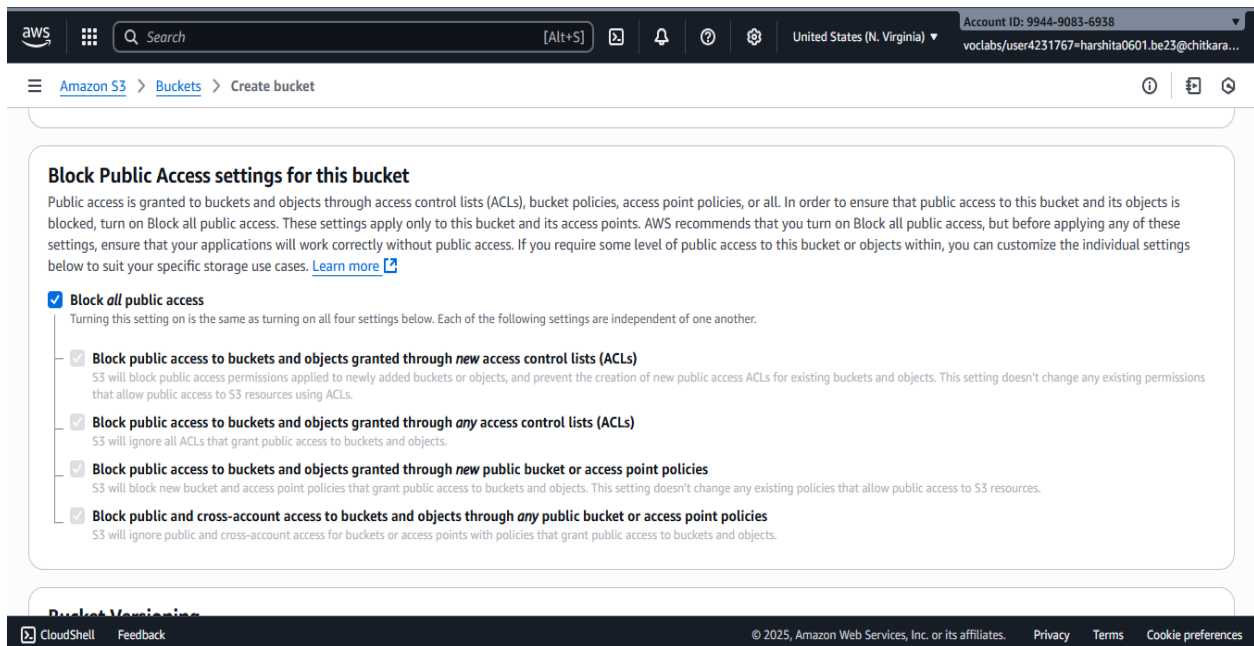


Figure 4.4

6. Click **Create bucket**.

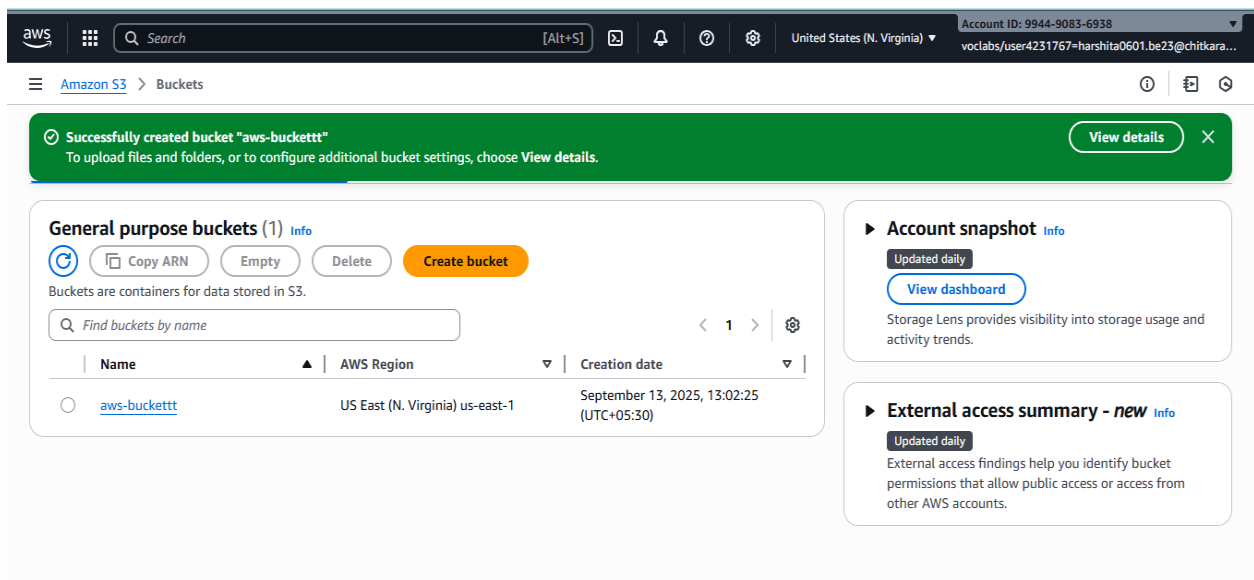


Figure 4.5

Step2: Upload Objects

1. Open your bucket.

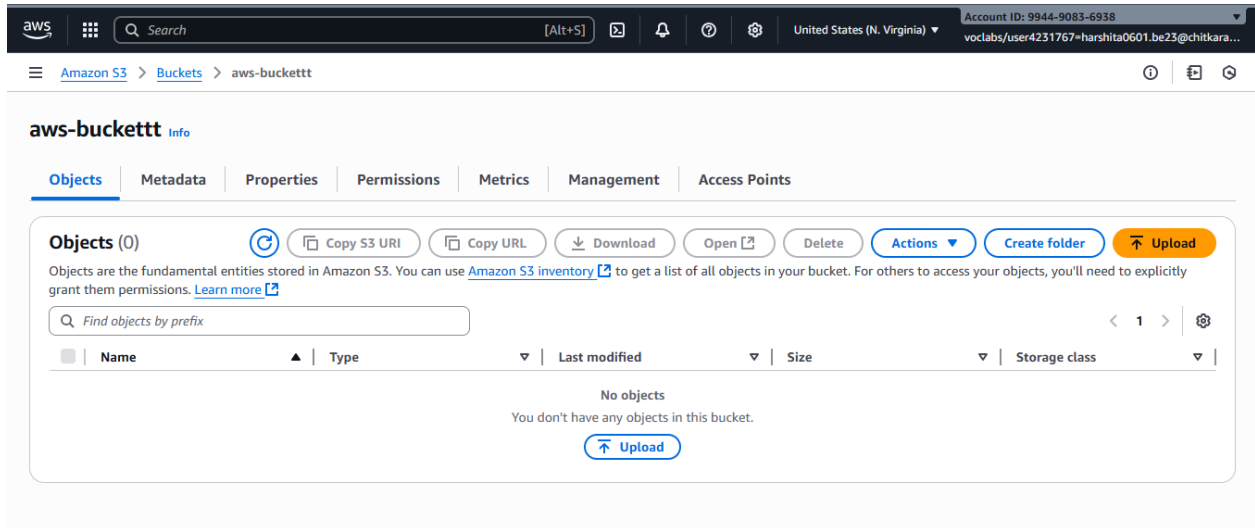


Figure 4.6

2. Select your files or pdf for uploading.

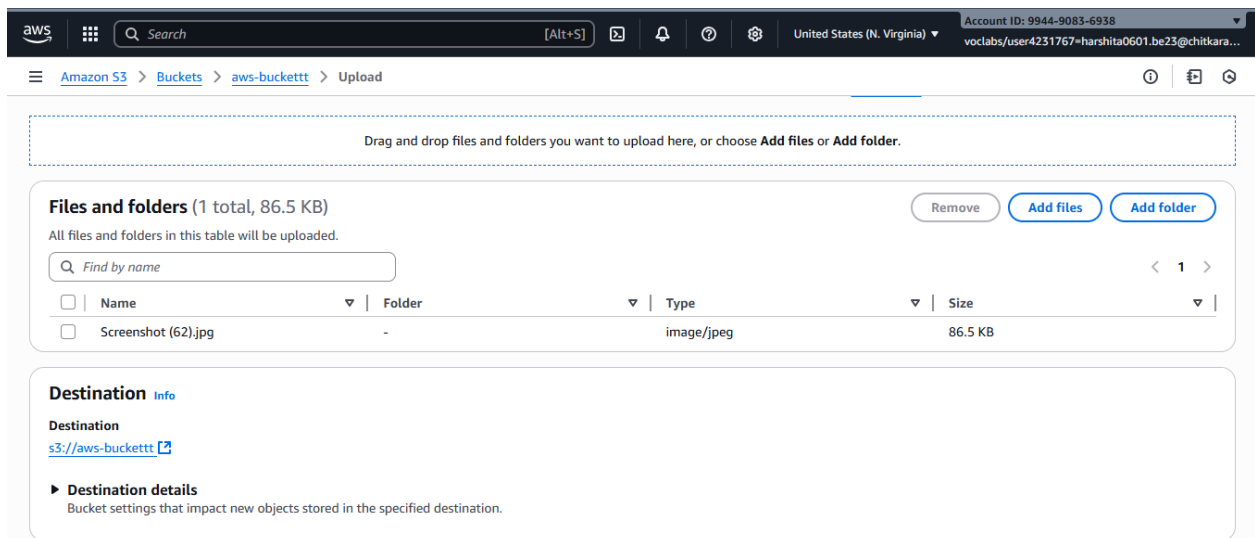


Figure 4.7

3. Click Upload.

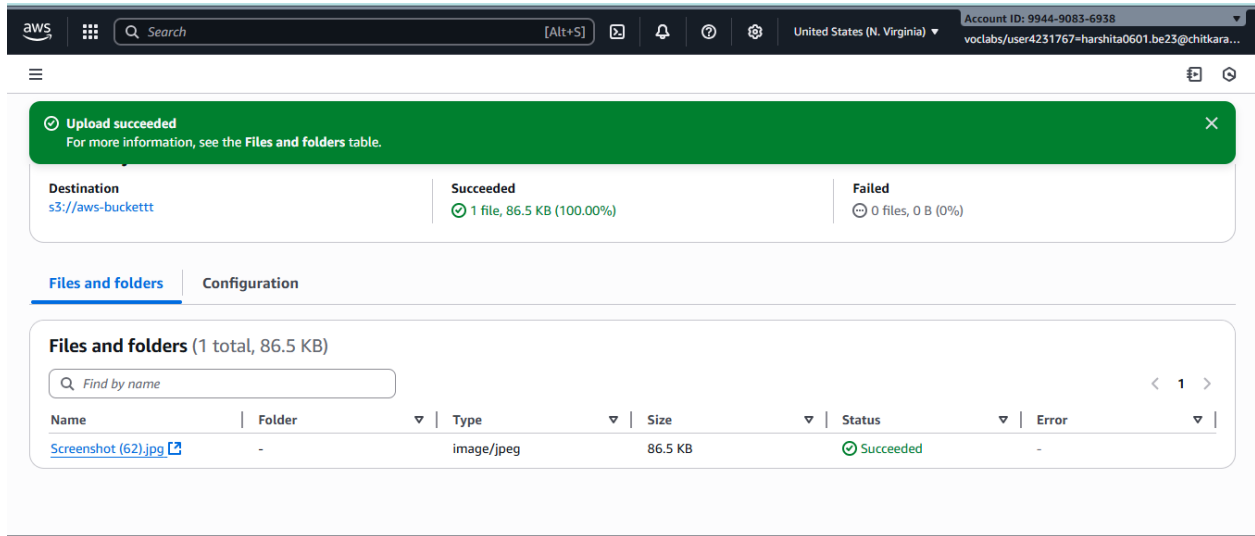


Figure 4.8

Step 3: Set Permissions with a Bucket Policy

Since you want users to access your files, you need to set a **bucket policy**.

1. Open your bucket → **Permissions** tab → **Bucket Policy** → **Policy Generator**.

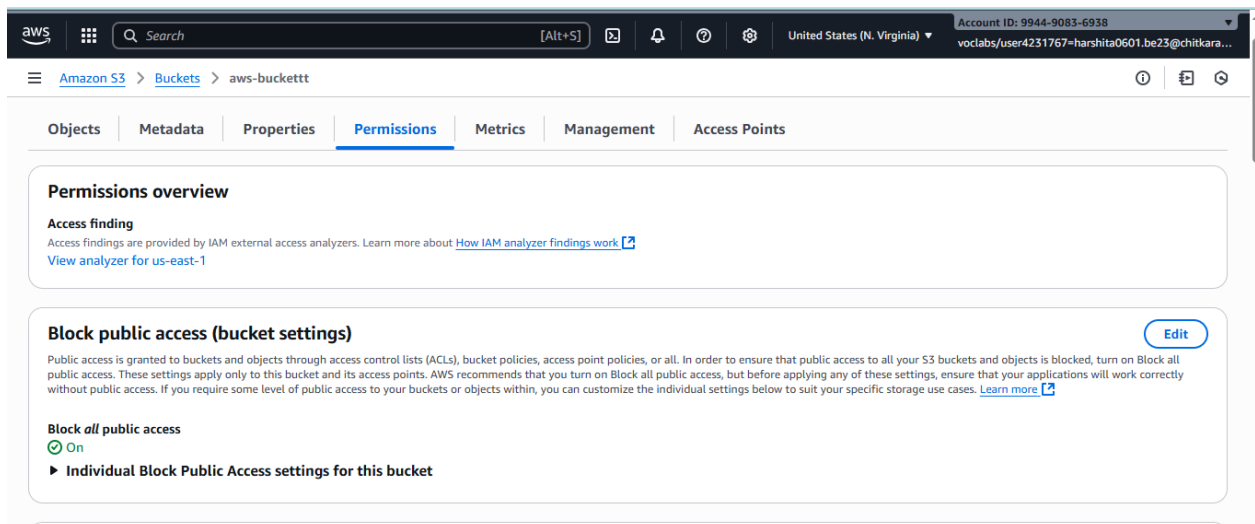


Figure 4.9

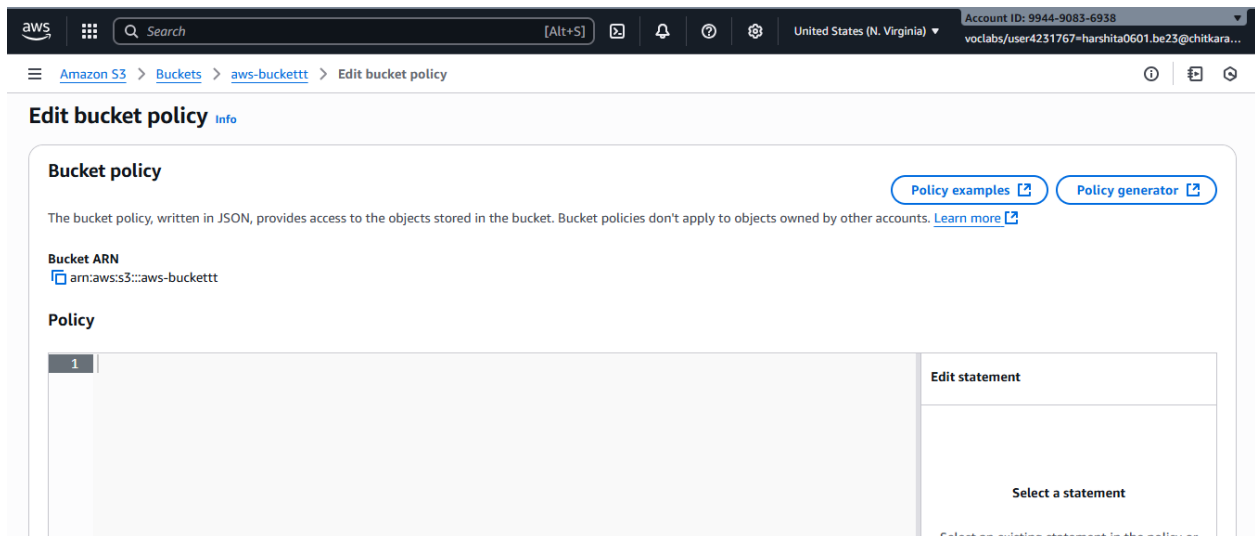


Figure 4.10

2. Select:

- **Policy Type:** S3 Bucket Policy
- **Effect:** Allow

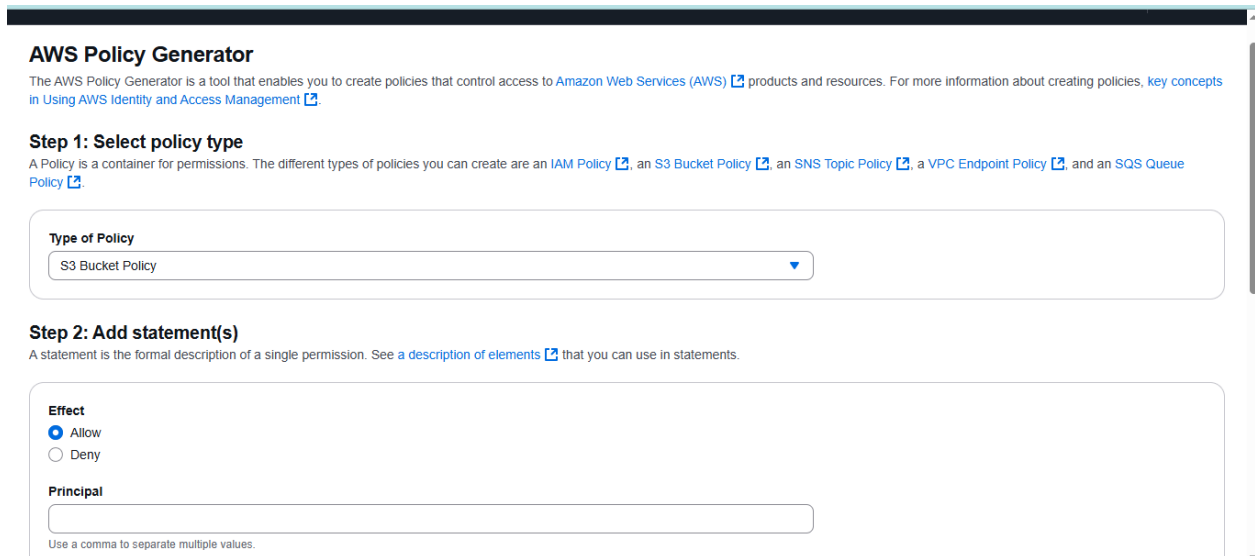


Figure 4.11

- **Principal:** * (means everyone)
- **Actions:** choose → GetObject
- **ARN:** all resources

☐ Deny

Principal

Use a comma to separate multiple values.

Actions
☐ All Actions (""")

Amazon Resource Name (ARN)
☒ All Resources (""")

Step 3: Generate policy

A policy is a document (written in the [Access Policy Language](#)) that acts as a container for one or more statements.

Figure 4.12

3. Click **Add Statement** → Then **Generate Policy**.

ARN should follow the following format: arn:aws:s3:::\${BucketName}/\${KeyName}. Use a comma to separate multiple values.

Statements added (1)

You added the following statements. Click the button below to Generate a policy.

Principal(s)	Effect	Action	Resource(s)	Condition(s)	Remove
*	Allow	s3:GetObject	*	None	Remove

Step 3: Generate policy

A policy is a document (written in the [Access Policy Language](#)) that acts as a container for one or more statements.

Figure 4.13

4. Copy the generated JSON and paste it into your **Bucket Policy editor**.

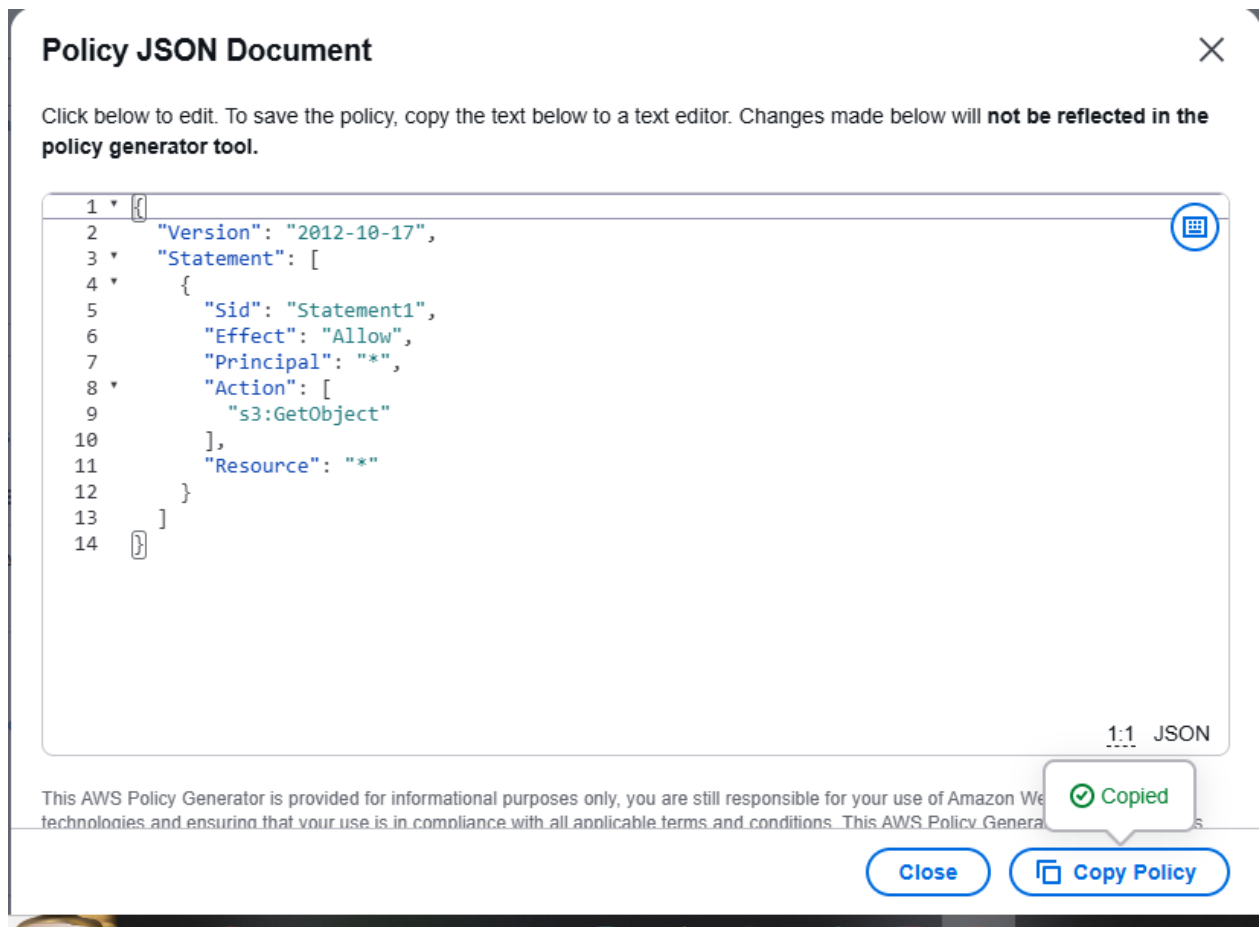


Figure 4.14

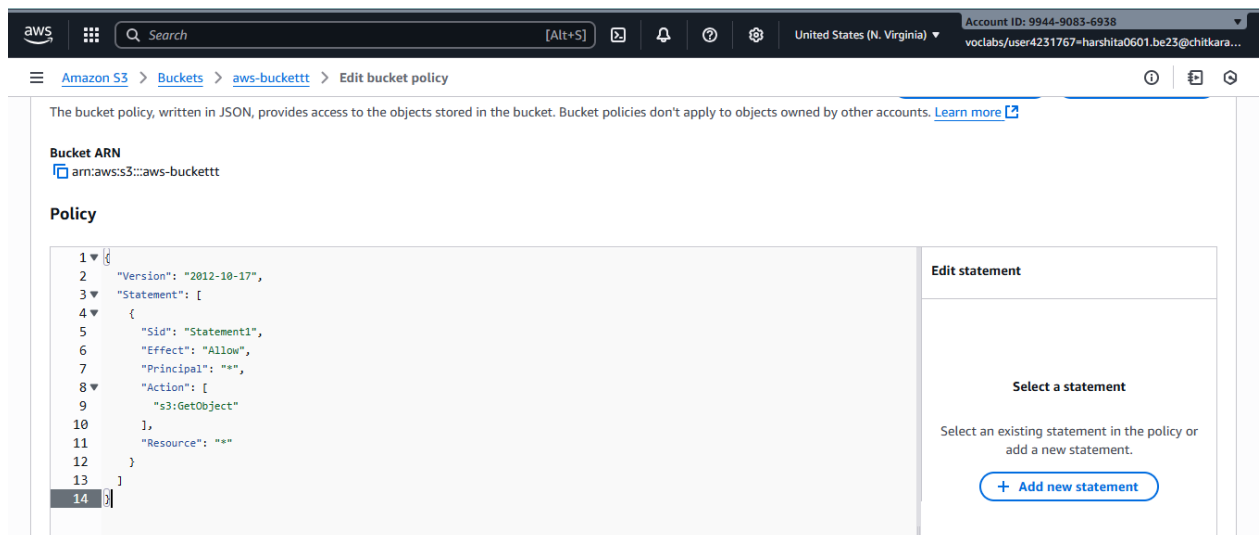


Figure 4.15

5. Save the policy :- this allows public read access to all the objects in your bucket.

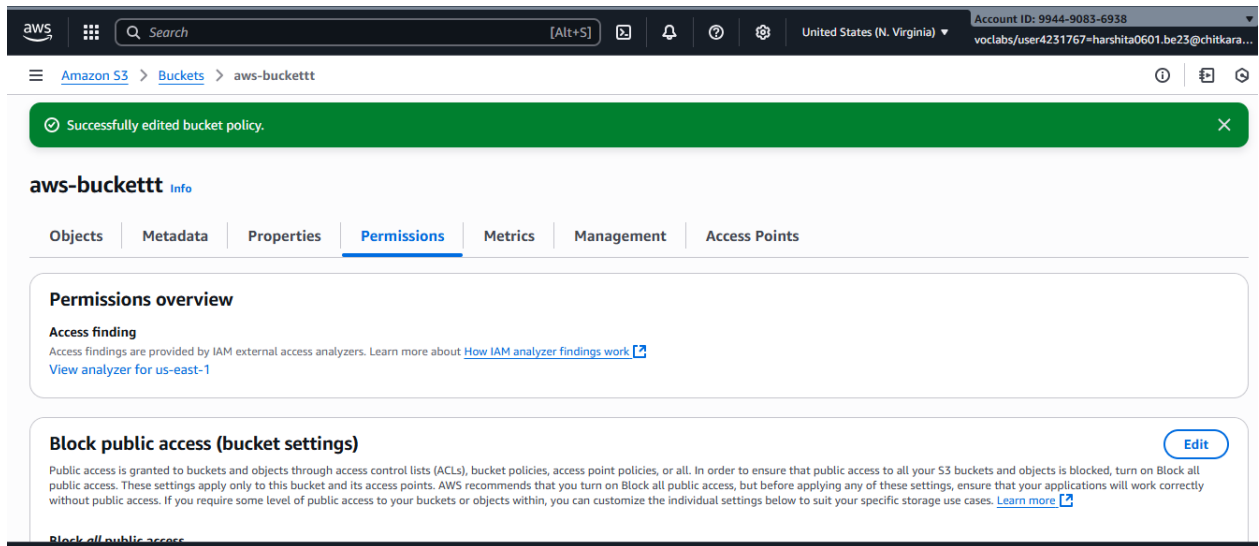


Figure 4.16

Step 4: Enable Static Website Hosting

1. Go to your bucket → **Properties** tab.

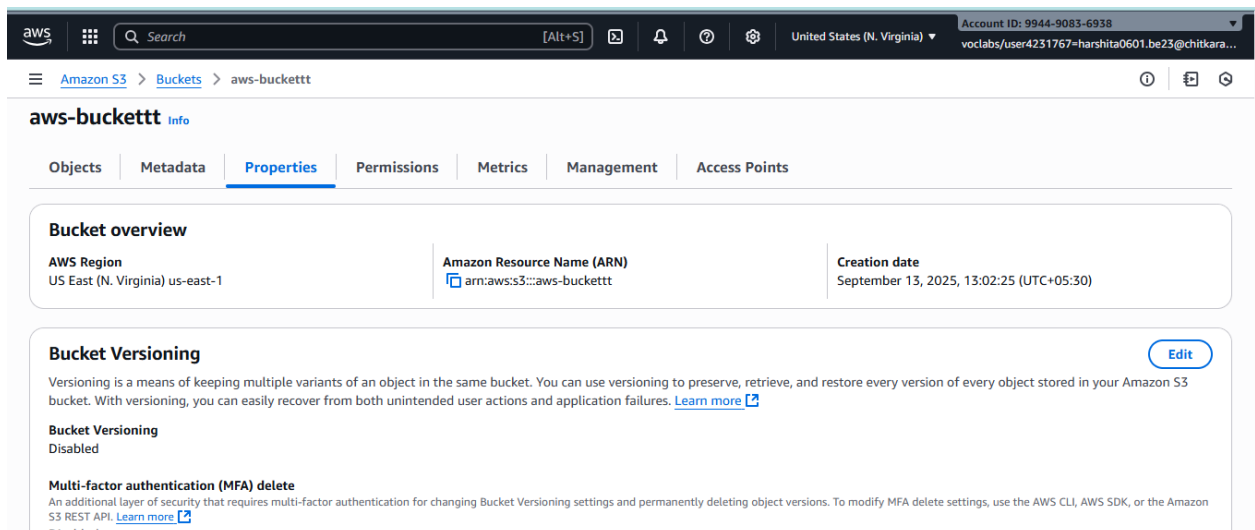


Figure 4.17

2. Scroll to **Static website hosting** → Click **Edit**.
3. Select **Enable**.
4. For **Index document**, type: index.html.
5. Save Changes.

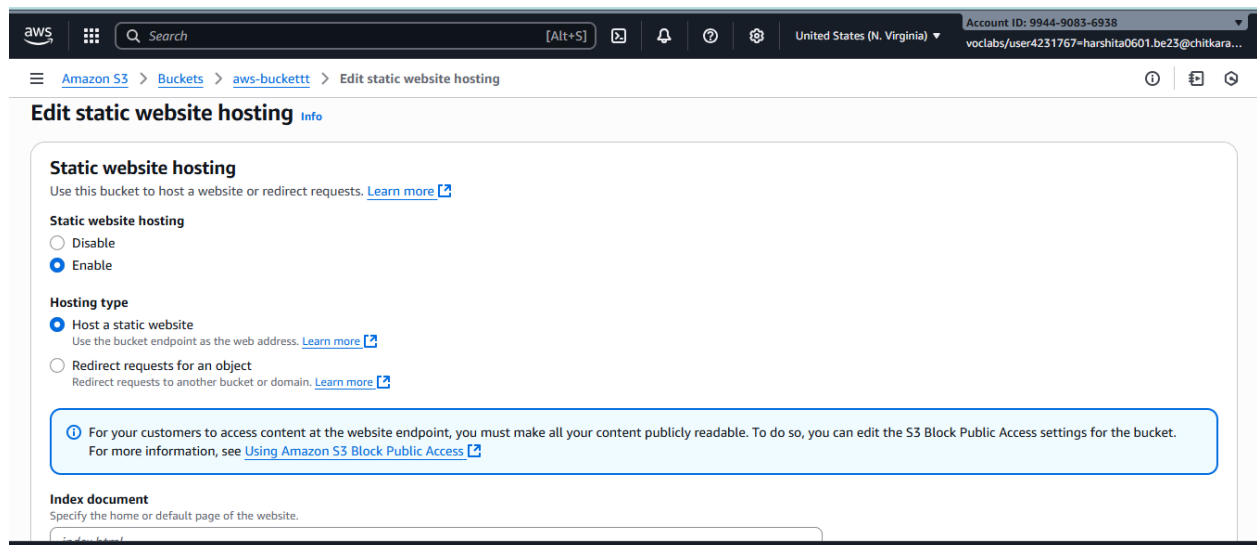


Figure 4.18

Step 5: Access Your Website

- After enabling hosting, AWS gives you a bucket website endpoint URL, something like:
<http://my-static-site-bucket.s3-website-us-east-1.amazonaws.com>
- Share this URL → Your static website is live!

Registration Form

Full Name
Enter your full name

Email
Enter your email

Password
Enter your password

Confirm Password
Confirm your password

Register

Figure 4.19



Learning Outcomes:

1. Created an Amazon S3 bucket and understood its role in cloud storage.
2. Uploaded and managed objects within an S3 bucket.
3. Configured bucket policies and access controls to secure data and manage permissions.
4. Enabled and tested static website hosting using Amazon S3.
5. Understood how S3 can be used for cost-effective, scalable, and highly available content delivery